

华中科技大学

《软件工程》项目报告

题目：大语言模型(LLM)下的文本智能纠错系统设计

课程名称：软件工程

专业班级：CS2110 班

组 名：生产大萝卜的队

同组成员：学号：U202115652

姓名：李嘉鹏

学号：U202115415

姓名：崔昊阳

学号：U202115641

姓名：高朗

学号：U202115630

姓名：裴俊博

学号：U202115647

姓名：王颖彬

指导教师：瞿彬彬

报告日期：2023 年 12 月 6 日

任 务 书

一 总体要求

1. 综合运用软件工程的思想，协同完成一个软件项目的开发，掌握软件工程相关的技术和方法；
2. 组成小组进行选题，通过调研完成项目的需求分析，并详细说明小组成员的分工、项目的时间管理等方面。
3. 根据需求分析进行总体设计、详细设计、编码与测试等。

二 基本内容

自行组队，针对一个选题完成相应的软件设计与开发。设计与开发中软件过程必须包括：

1. **问题概述、需求分析：**正确使用相关工具和方法说明所开发软件的问题定义和需求分析，比如 NABCD 模型，Microsoft Visio，StarUML 等工具 (20%)；
2. **原型系统设计、概要设计、详细设计：**主要说明所开发软件的架构、数据结构及主要算法设计，比如墨刀等工具 (35%)；
3. **编码与测试：**编码规范，运用码云等平台进行版本管理，设计测试计划和测试用例 (30%)；
4. **功能创新：**与众不同、特别吸引用户的创新 (10%)；
5. **用户反馈：**包括用户的使用记录，照片，视频等 (5%)。

目 录

任务书.....	I
1 问题定义.....	4
1.1 项目背景与意义.....	4
1.1.1 项目 NABCD 模型.....	4
1.2 项目基本目标.....	7
1.3 可行性分析.....	8
1.3.1 国内外同类型项目研究现状.....	8
1.3.2 项目可行性分析.....	9
1.4 人员管理和项目进度管理.....	10
2 需求分析.....	14
2.1 需求分析概述.....	14
2.2 原型系统设计.....	15
3 概要设计和详细设计.....	19
3.1 系统结构.....	19
3.1.1 总述.....	19
3.1.2 文本纠错模型.....	20
3.1.3 后端.....	21
3.1.4 前端.....	22
3.2 关键数据结构设计.....	24
3.2.1 后端数据结构设计.....	24
3.2.2 前端数据结构设计.....	24
3.2.3 模型端数据结构设计.....	25
3.3 关键算法设计.....	27
3.4 数据管理说明.....	28
3.4.1 用户数据.....	28

3.4.2 用户登录状态.....	28
3.4.3 用户文件存储.....	29
4 实现与测试.....	30
4.1 实现环境与代码管理.....	30
4.1.1 系统实现环境.....	30
4.1.2 代码管理.....	30
4.2 关键函数说明.....	32
4.2.1 M2 Scorer 评测模块关键函数说明.....	32
4.2.2 ChatGLM-6B 大语言模型关键函数说明.....	33
4.2.3 前端关键对象说明.....	34
4.2.4 后端关键函数说明.....	35
4.3 测试计划和测试用例.....	36
4.3.1 黑盒测试.....	36
4.3.2 面向对象软件测试.....	37
4.4 结果分析.....	38
5 总结.....	43
5.1 用户反馈.....	43
5.2 全文总结.....	45
5.3 小组会议照片与会议记录.....	47
5.4 项目获奖.....	49
6 体会.....	50
参考文献.....	55
附录.....	56
附录 A 模型端关键代码.....	56
附录 B 前端关键代码.....	61
附录 C 后端关键代码.....	64
附录 D M2 Scorer 关键代码.....	68

1 问题定义

1.1 项目背景与意义

当前，随着科技创新和经济发展的不断加快，越来越多的人开始投入到学术研究中，国际学术圈的繁荣发展也离不开海量学术论文的发表与交流。然而，在撰写论文时，作者很容易因粗心等原因面临中文拼写错误（Chinese Spelling Correction, CSC）和语法错误（Grammatical Error Correction, GEC），这些错误常常会给读者带来困惑，影响论文阅读效果。若仅仅依靠人眼识别错误，则效率较低，且往往会遗漏一些错误。当前市面上主流的学术论文检错网站都主要采用传统深度学习模型，在学术论文这一细分领域上表现出的性能仍然有提升空间。

因此，我们提出这个项目，旨在开发一款文本智能纠错系统，帮助用户解决拼写错误和语法错误，为学术界提供更加高效的论文检错平台。

1.1.1 项目 NABCD 模型

根据上述分析，我们的系统需要对语料中的中文拼写错误和语法错误分别作出合理的判断和修改。项目的 NABCD 模型如下。

（1）N（Need 需求）

本软件的主要用户包括科研人员、研究生等中文论文写作人士。这些用户往往需要大量的时间对论文进行拼写和语法纠错，所以需要有一个能够自动检测和纠正论文错误的工具来提高论文质量，减少改错花费的时间。为此，本软件需要能自动检测和标识论文中的拼写错误（包括标点符号错误）、语法错误以及其它常见的语法问题。作为进阶功能，我们还希望这一软件可以修正论文的格式和语言风格，使其更加符合学术类文本的特征，提高用户体验。

中文拼写纠错旨在检测和纠正中文文本中的拼写错误。它是各种自然语言处理领域下游应用的关键研究领域，如光学字符识别、搜索查询校正和论文评分。然而，CSC 非常具有挑战性，因为它需要处理相互混淆的字符，例如在语音和视觉上相似的字符。例如，“素（sù，朴素）”和“诉（sù，起诉）”由于有共同的发音“sù”，彼此是混淆的字符。图 1-1 展示了 CSC 问题的形式以及学术界普遍采用的处理范式。

Input	我这一次写信给你是想跟你安排一下关(guān)以(yǐ)我们要见面的。
baseline	我这一次写信给你是想跟你安排一下所(suǒ)以(yǐ)我们要见面的事。
Ground Truth	我这一次写信给你是想跟你安排一下关(guān)于(yú)我们要见面的事。
Translation	I am writing to you this time to make arrangements with you about our meeting.

Input	为了减少急遍(biàn)的生孩子率，需要呼吁适当的生育政策。
baseline	为了减少急速(sù)的生孩子率，需要呼吁适当的生育政策。
Ground Truth	为了减少急变(biàn)的生孩子率，需要呼吁适当的生育政策。
Translation	In order to reduce the rapidly changing rate of childbirth, it is necessary to call for an appropriate childbirth policy.

图 1-1 中文拼写纠错（CSC）示例

语法错误纠正（GEC）旨在自动检测和纠正文本中的语法相关错误。它在写作助手、语音识别系统等实例中的实际应用引起了人们的关注。下图展示了 GEC 问题在近年来的一种解决范式。

Character Replacement		
Input: 我没有权利破害别人的生活 (En: I have no right to destroy other people's lives.)	Label: 坏	Prediction: 坏 (99.97%)
Input: 代沟问题越来越深刻。 (En: The problem of generation gap is getting worse.)	Label: 严重	Prediction: 严 (79.94%) 重 (91.85%)

Character Insertion		
Input: 吸烟不但对自己的健康好，而且对非吸烟者带来不好的影响。 (En: Smoking is not only bad for your health, but also bad to non-smokers.)	Label: 不	Prediction: 不 (99.98%)
Input: 我下次去北京的时候，一定要吃北京烤鸭，我们在北京吃过的 是越南料理等外国的。 (En: Next time I go to Beijing, I can not miss the Peking Duck. What we have eaten in Beijing are Vietnamese cuisine and other foreign dishes.)	Label: 饭菜	Prediction: 美 (40.66%) 食 (33.55%)

图 1-2 中文语法纠错（GEC）示例

(2) A（Approach 做法）

我们的方案主要基于大语言模型 LLM 中的 ChatGLM-6B 模型。我们会首先生成一个高质量的学术论文数据集（涵盖拼写错误、语法错误和逻辑错误等多种错误类型）。获得高质量数据集后，我们将对 ChatGLM-6B 模型进行优化和微调（finetune），使其能够更好地识别和纠正数据集中的错误。对于每轮微调，我们会使用目前国际上通用的 M2 评分标准判定模型的效果，进而调整模型的参数。

在系统层面，我们将开发一个用户友好的、具有高精确度和解释性的中文科研论文智能纠错系统，通过精心设计用户界面（UI）和使用逻辑，给用户带来优秀的使用体验，降低学术界的论文纠错成本，提高科研效率。

（3）B（Benefit 好处）

本项目可以为用户带来多维度的帮助，详细列举如下。

①前端

- 用户友好性：前端能接受多种格式的文本，可以满足不同用户的需要。
- 实时性：前端可以快速将用户输入的文本传递给后端进行进一步处理，实现即时的纠错反馈。

②后端

- 强大的计算能力：后端可以部署在高性能服务器上，利用 LLM 进行大规模、高效的文本纠错处理。
- 完善的用户注册与登录功能：后端与数据库连接，及时将用户的注册信息存储，同时运用 flask_login 维护登录状态。用户相关信息加密保存在后端数据库中，确保安全性。
- 可扩展性：后端可以根据需要灵活地调用不同版本的 LLM，以适应不同的纠错需求或使用全新的前沿模型。

③模型端

- 广泛的纠错类型处理：LLM 在处理纠错任务时具有通用性，不仅适用于拼写错误，还可以发现并纠正语法、格式、标点符号等各种文本错误。
- 不断优化的性能：LLM 可通过大规模的训练数据进行学习，不断优化纠错能力，并在实际应用中逐渐提升准确性。针对当前对论文方面纠错的应用较少的状况，LLM 将会学习大量的学术论文集，提高对论文的纠错水平。
- 上下文感知能力：LLM 在训练过程中可以通过学习大量语言数据来理解上下文信息（包括内容、行文思路、语言风格等），并对不同的语法结构、语义关系进行建模，更准确地纠正文本中的错误。

（4）C（Competitors 竞争）

与本软件类似的产品有百度公司推出的“百度 AI 文本纠错”平台。其优势在于能广泛支持各种类型文本的纠错，并能完成实时纠错，提供一定的纠正建议。但其劣势在于会发生一定程度的误判，在某些专业领域的特定术语和行业用语上

的纠错能力也相对较弱。不仅如此，其对大文本的处理能力也较弱，无法满足对学术类长文本的纠错要求。

(5) D (Delivery 推广)

推销“大语言模型下的文本智能纠错系统”主要需要考虑以下三个方面。

● 产品推广

①确定目标客户群体：确定潜在客户，例如企业、学术机构、写作工作者等。

②开展市场调研：了解行业需求与竞争对手情况，为产品定位和差异化提供准确依据。

③制定营销策略：包括线上线下宣传推广、参加行业展会或研讨会、推出针对目标客户的定制化方案等。

● 销售

①提供充分的产品信息：准确描述产品特点、功能优势和性能指标，帮助客户理解产品价值。

②建立合作关系：与潜在客户进行洽谈，提供产品演示，解答相关疑问，以期达成合作意向。

③提供灵活的销售模式：根据客户需求，提供订阅、按量计费、定制化服务等多种销售模式。

● 用户培训及系统支持维护

①提供培训课程和教材：帮助用户了解产品的使用方法和操作流程。

②提供技术支持和解决方案：通过在线客服、电话支持等渠道，及时回答用户疑问、解决问题。

③定期更新和升级：及时修复系统漏洞，持续改进系统性能和功能，提供更好的用户体验。

1.2 项目基本目标

根据上述分析，本项目旨在研发并完善一种主要基于大语言模型的文本智能纠错系统，针对学术论文领域中常见的拼写错误和语法错误问题提出解决方案。本项目将生成高质量的学术论文数据集，以 ChatGLM-6B 模型为基础，通过对其进行预训练和微调，实现学术论文的自动检错和纠错功能，同时提高模型的效

率和准确性。我们也会同步进行前后端的搭建，并实现其与模型端的接入，打造简洁明了的用户界面。

1.3 可行性分析

我们首先调研了国内外同类型项目的研究现状，掌握了目前文本纠错领域的大致样貌，然后从技术与经济两方面分析了项目的可行性。

1.3.1 国内外同类型项目研究现状

(1) 训练数据层面

当前，中文语病语料库存在的主要问题有二：总体样本数量较小、数据集精细化程度不高。

目前公开的中文文本错误数据集还不充分，尤其是针对具体错误类型的数据集。并且这部分数据集的语料取材大多较为广泛，没有针对具体错误类型进行深入研究。

第一，针对拼写纠错模型的训练，相关研究主要选用了最新的 SIGHAN 基准数据集。这是一个存在 22000 条错误样例的文本数据集，包含 461 种类型的拼写错误。它的主要问题是精细化程度不高，只能满足通用纠错模型的训练，在科研论文这种修改限制高、拼写错误隐蔽的领域不能体现很好的测试效果。

第二，现有语法错误数据集普遍较少。对于本项目所涉及的中文学术类文本纠错，目前还缺乏能准确表现论文常见语法错误的自动生成样本算法。因此首先我们采用人工标注的方法生成一个小规模高质量的数据集；接着采用 self-instruct 方法利用 ERNIE 4.0 模型进行数据集自动生成，在保证数据集质量的基础上扩大数据集的规模，使之可以用于 ChatGLM-6B 模型的微调。

(2) 模型与算法层面

近年来，随着机器学习算法模型不断改进，文本纠错问题取得了显著进展。2022 年，Zhengxiao Du 等提出了自回归空白填充预训练的通用语言模型 (GLM)，该模型通过添加二维位置编码和允许任意顺序预测跨度，改进了空白填充预训练，从而在自然语言理解、条件生成和无条件生成的任务中，在相同模型大小和数据的情况下优于 BERT、T5 和 GPT 等大模型，并且在语句纠错方面表现出很大潜

力。这为本研究选择 ChatGLM-6B（清华大学研发的开源 GLM 模型）作为文本纠错模型基准来进行改进和微调提供了充分理论依据。GLM 具体运行机制可参考图 1-3。

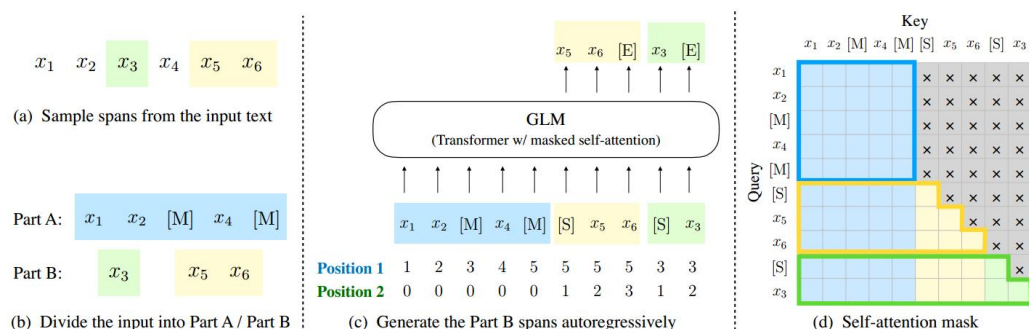


图 1-3 GLM 模型运行机制示例

1.3.2 项目可行性分析

为了确保项目开展顺利，我们从技术和经济两方面分析了项目的可行性。

(1) 技术可行性

本项目要求实现文本智能纠错，需要即时输出反馈结果，因此需要较高的算力。目前，我们项目拥有 8 张 NVIDIA GeForce RTX 3090 显卡，具有充足的算力资源，同时还拥有 20000 元经费，可用于人工标注数据集、云端服务部署等必要开销。同时，为了呈现出简单易用的用户界面，还需要进行前后端的设计与对接。

团队成员均为计算机学院本科生，具有丰富的计算机领域专业知识，如数据挖掘、大数据分析、机器学习、深度学习、前后端应用开发、软件测试与迭代等，在项目推进过程中具有良好的团队凝聚力和共同目标。团队成员积极参与创新创业实践，并取得了丰富成果，该项目也被成功立项为 2023 年国家级大学生创新创业训练计划项目，具有完备的组会交流制度。

针对本项目，团队成员已有的技术积累包括 VS Code 的使用、远程服务器的 ssh 连接与使用、Python / C++ / TypeScript 多语言编程、大模型微调、前后端开发（包括 vue、flask 架构等）等，具有合作设计开发项目系统并进行调试的专业能力。

目前项目已产出第一代可用系统，证明本项目具有充分的技术可行性。

(2) 经济可行性

本项目的定位是一种方便科研人员进行论文修改的 WEB 应用。在开发完成后，我们将会首先进行小范围推广，扩大项目的知名度与影响力，并首先在校园内启动试用工作。在一段时间的试用期后，我们将针对性地收集用户反馈，并对系统进行优化后再次上线。

由于目前市面上小型服务器的租赁成本并不高，因此本项目可以长期在云上进行运营，并通过流量变现等方式获得回报。同时，对于人工标注数据集的工作，可以考虑在年级大群中招募志愿者，以相对合理的价格获得高质量的标注结果。因此总体来说，本项目的收益/投资比较高。

综合上述分析，我们认为本项目的开发是必要且可行的。

1.4 人员管理和项目进度管理

通过系统地分析项目目标，本小组共同商议并制定了合理的人员分工安排与项目进度计划，通过对任务设置阶段性期限，确保项目有序推进。

项目的开发主要分为四个部分：

- 前期调研阶段
- 初步探索阶段
- 分模块实现阶段
- 试运行、测试和正式上线阶段

本项目组共包含 5 名成员，在不同阶段的不同任务中，详细人员分工安排如表 1-1 所示。（为了简化下表，用字母代表项目成员：A-李嘉鹏，B-崔昊阳，C-高朗，D-裴俊博，E-王颖彬）

表 1-1 软件工程项目人员分工安排

阶段	任务		负责人
前期调研阶段	模型侧	阅读语法纠错的前沿论文，了解现有工作的优缺点	AC

		查找用于中文语法纠错的数据集	C
		阅读论文，查找语法错误数据集的生成方法	BC
		爬取论文，构建正确句子数据集	C
		阅读 LLM 论文，对 LLM 形成初步了解	AC
	前后 端侧	进行系统的总体设计，完成需求分析 UML、 系统架构图等	ABCDE
		学习前端设计和前后端交互相关知识	D
		学习后端相关知识（Python、Flask 框架等）	E
		进行前端页面的初步设计	D
初步探索 阶段	模型 侧	阅读语法纠错任务评价指标前沿论文，找到一个 好的评价指标并跑通测试代码	A
		对比不同的数据集生成算法，找出效果较好的 算法	B
		部署 ChatGLM-6B 模型，并熟悉其使用	BC
		直接使用 ChatGLM-6B 模型进行语法纠错，查 看其效果	AC
		阅读有关模型微调的前沿论文并进行实践，查 找效果最好的微调方法，使用 M2 标准进行评 测	A
		尝试用一些微调算法微调 ChatGLM-6B 模型， 查看效果并对比	C
	前后 端侧	研究学习使用开源编辑器框架 tiptap	D
		调研文档格式转换的方法	DE
分模块实现 阶段	模型 侧	使用前期找到的方法对 ChatGLM-6B 模型进 行微调	B
		人工标注小规模的高质量数据集	ABCDE
		撰写中文学术类文本纠错标注手册	AB
		使用前期找到的数据集生成方法生成大规模 数据集用于模型微调	B

		编写使用微调好的模型进行语法纠错的函数， 便于展示系统使用	BC
		将整个模型在服务器上进行正式部署	C
		与后端配合实现展示系统的后端搭建	C
	前后 端侧	实现数据库操作功能	E
		实现基本界面框架	D
		实现用户验证相关的后端支持	E
		实现用户登录注册界面	D
		实现个人信息和文件管理界面	D
		实现基本的编辑器界面并调用模型进行纠错	D
		完善编辑器功能，支持多种富文本样式	D
		实现文件管理功能的后端支持	E
		实现文档格式转换	DE
	进行系统联调		AB
试运行、测试和正式上线阶段	对系统进行测试，并改正发现的 Bug		AB
	系统正式上线准备工作		ABCDE
	系统正式上线运行		ABCDE

为了进行项目整体进度管理，我们引入甘特图对任务的具体时间进行分配，如图 1-4 所示。其中每一行代表一个子任务，最左侧为该子任务的负责人。

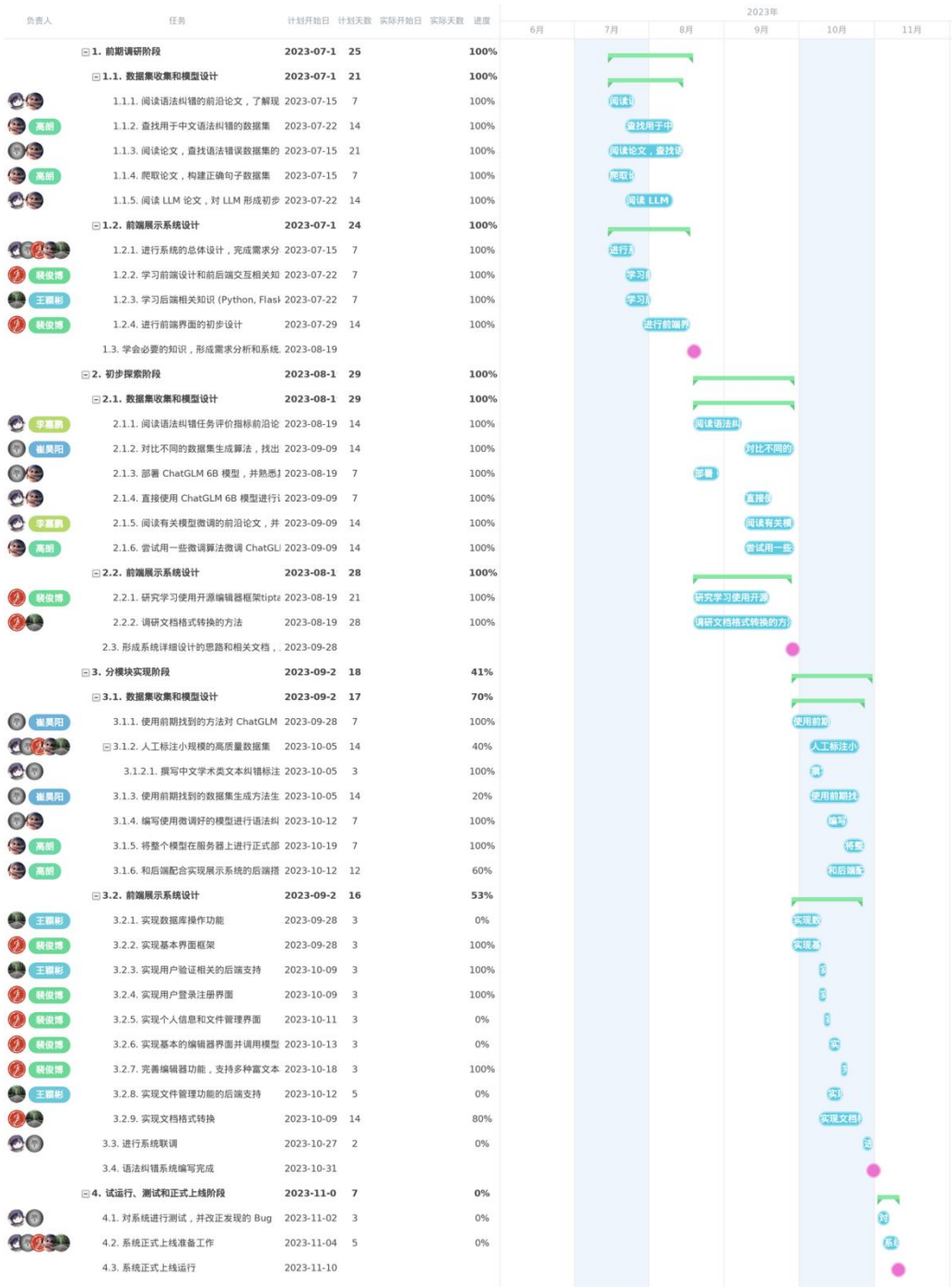


图 1-4 项目甘特图

2 需求分析

2.1 需求分析概述

经过前期的头脑风暴讨论，团队确定了项目开发的总体方向，思维导图 Mindmap 如图 2-1 所示。

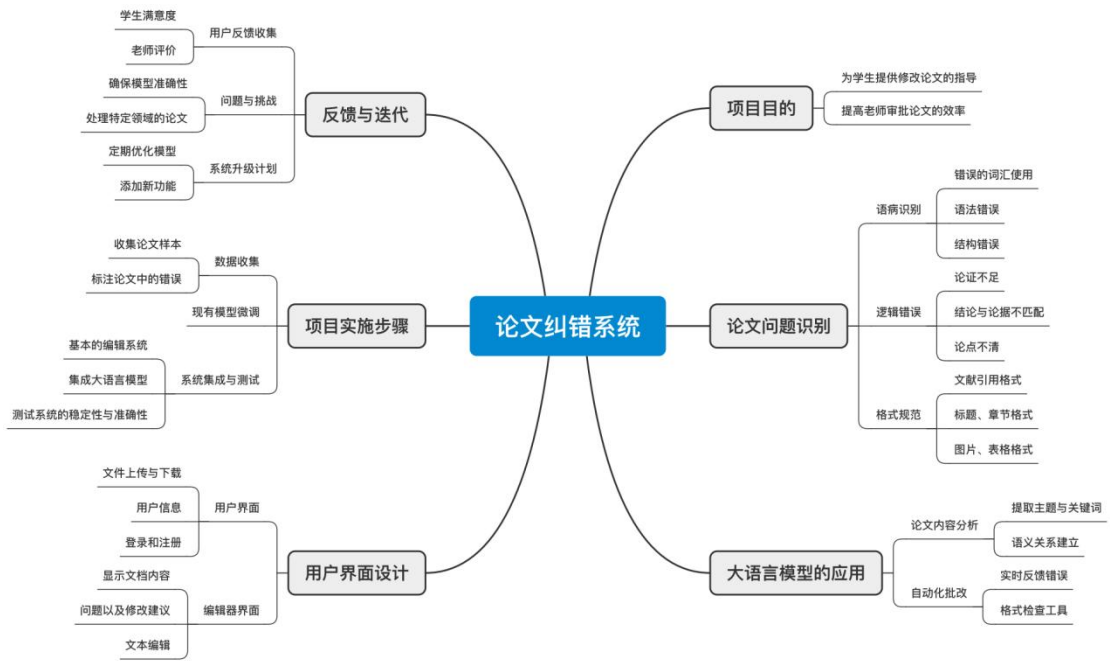


图 2-1 头脑风暴 Mindmap

● 用户需求

用户对本系统的需求包括注册账号、上传文件、根据错误识别提示作出接受或不接受操作、保存或下载文件、查看个人中心等。

● 功能需求

本系统需要完成多项功能，包括用户基本信息登录与注册、文本的上传与下载、文本内容解析、文本编辑器实时显示文档内容和标注错误位置、实时文本编辑、文本存储、设置操作教程等。为了让用户获得良好的使用体验，需要设计简洁明了的用户界面，同时具有良好的引导机制。

● 非功能需求

①效率：本系统需要对用户输入进行快速识别与反馈，检错时间控制在 3 秒以内；

②准确性：本系统返回的检错结果需要是准确可靠的，对于典型病句，期望达到 70%以上的错误识别率；

③安全性：本系统将用户注册时提供的用户名、密码等隐私信息加密存储在后端数据库中；

④可移植性：本系统分别设计模型端、前端、后端三个部分，彼此之间相互联系，基本功能函数采用模块化管理，同时设置了完备的部署教程，具有良好的可复用性与可移植性。

2.2 原型系统设计

在正式进行开发工作前，我们运用墨刀工具设计了项目的原型系统（链接：https://modao.cc/proto/slY3rTVjs28orgmrZPxNN/sharing?view_mode=read_only），主要分为“首页”、“登录页面”、“个人中心”、“编辑器”、“团队介绍”五个部分，以便于更准确直观地说明主要功能和用户交互界面，如图 2-2 所示。

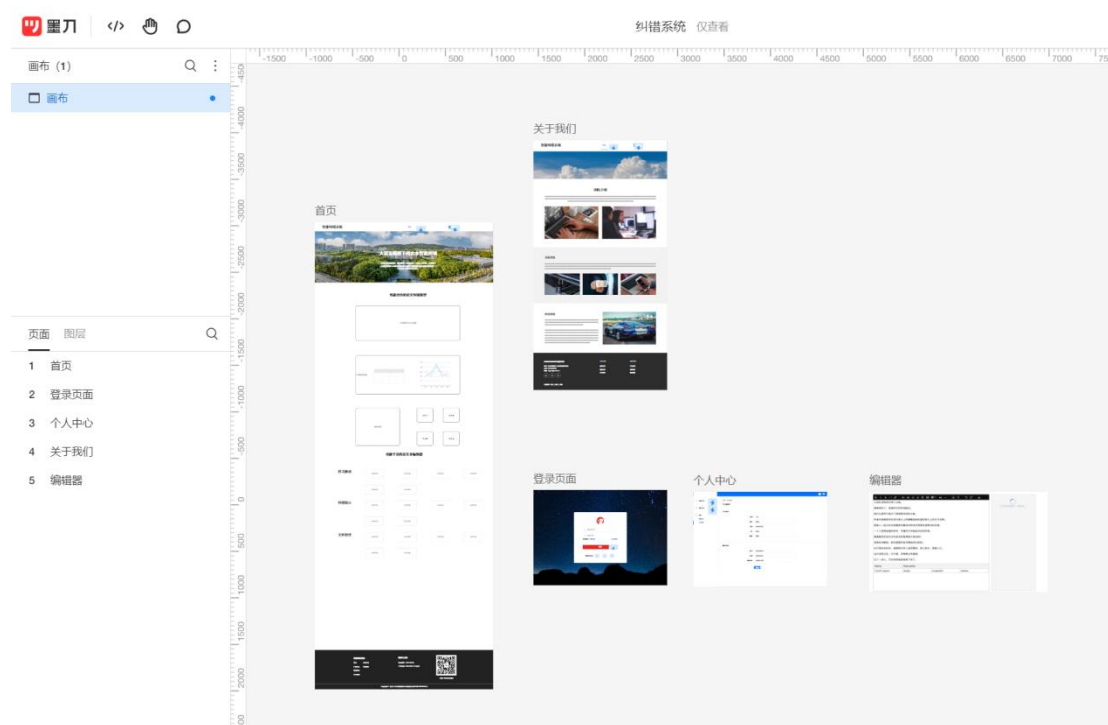


图 2-2 原型系统总体架构

上述各部分的原型系统设计分别如图 2-3~图 2-6 所示。



图 2-3 “首页”部分原型系统设计

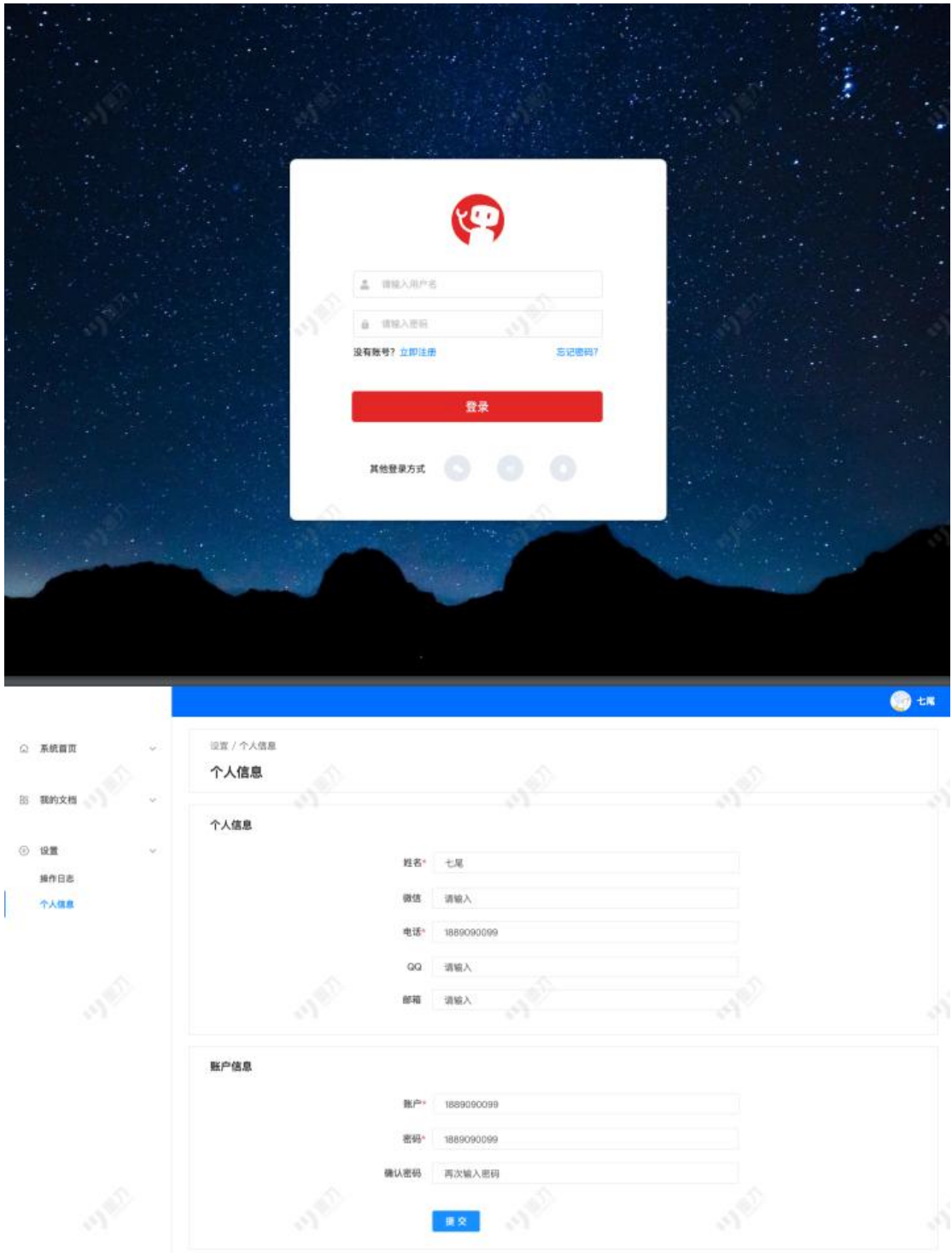


图 2-4 “登录页面”、“个人中心” 部分原型系统设计



图 2-5 “编辑器”部分原型系统设计



图 2-6 “团队介绍”部分原型系统设计

3 概要设计和详细设计

3.1 系统结构

3.1.1 总述

本系统以一个WEB应用的形式实现，采用了前后端分离的架构。在这种架构中，前端负责处理用户界面和用户交互，而后端负责处理数据存储和业务逻辑，因此可以将应用程序的前端和后端功能分开开发、部署和维护。前后端分离架构是一种较为先进的WEB应用架构，它的几点优势如下：

- 独立开发

前端和后端可以由不同人员独立开发，提高了开发效率和并行开发能力。

- 可维护性

前端和后端的代码分离，使系统的维护更加容易，可以针对性地修改和更新其中的一部分。

- 扩展性

前后端可以分别进行水平扩展，根据实际需求增加前端或后端的实例数量，提高系统的性能和可伸缩性。

- 技术选型灵活

前后端独立演进，可以选择最合适的技术栈和工具，不受限于单一技术栈。

- 高可用性

前后端分离使得前端应用程序可以以静态文件的形式进行部署，通过内容分发网络实现不同区域间的快速访问。

为了支持前后端分离架构，本系统的前端使用 Vue 框架、node.js 运行环境和 TypeScript 语言进行开发。同时，为了便于模型的部署，后端使用 Flask 框架和 Python 语言开发。

由于大语言模型推理的计算量较大但所需的辅存空间较小，而用户信息存储等其它模块计算量较小但所需存储空间较大，所以本系统的后端分成两个部分。系统的大语言模型单独部署在具有 8 块 NVIDIA GeForce RTX 3090 GPU 的运算

服务器上，并以 API 调用的方式为后端的其它部分提供服务。系统的其它部分以及前端应用，例如进行用户身份鉴定和用户文档存储等的模块，部署在磁盘空间为 9.1 T 的存储服务器上。

本系统的模块关系如图 3-1 所示。

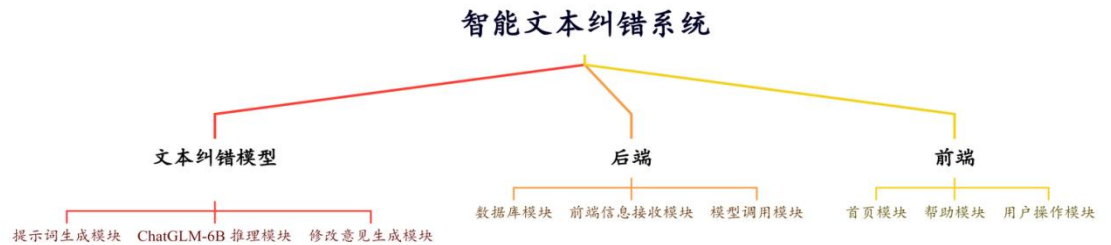


图 3-1 系统模块关系树

3.1.2 文本纠错模型

本小节主要介绍文本纠错模型的整体结构。

在本系统中，我们使用 ptuning 后的 ChatGLM-6B^[1]模型进行用户上传的文档拼写和语法错误的纠正。其中 ptuning 所用的大规模标注数据集采用 self-instruct^[2]方法，由 ERNIE-Bot 4.0 模型自动生成。

文本纠错模型分为 3 个主要功能模块：提示词生成模块、ChatGLM-6B 推理模块以及修改意见生成模块，如图 3-2 所示。

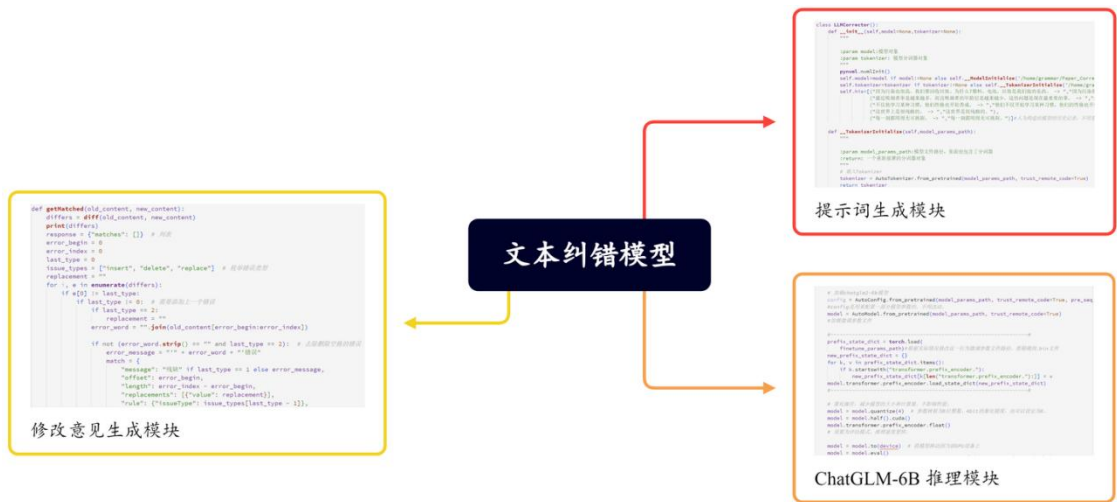


图 3-2 文本纠错模型系统结构图

提示词生成模块用于根据后端调用 API 时传入的句子来生成输入 ChatGLM-6B 模型的提示词。我们的提示词由任务类型、范例以及输入实例构成，

具体的提示词模板如下：

```
prompt = """中文语法纠错任务：将一段中文句子进行修正，如果句子没有明显语病则
返回原句。

    下面是一些范例：

    学生大概做飞机去北京。 -> 学生大概坐飞机去北京。 \n

    我觉得他很大胆，他不去外国。他觉得如果要改转变中国，所以就要从中国做
    的研究。 -> 我觉得他很大胆，他不去外国。他觉得如果要改变中国，就要在中国做研
    究。 \n

    吸烟者反对这样的措施。 -> 吸烟者反对这样的措施。 \n

    请对下述句子进行修正。返回你修改后的句子，无需其它说明和解释。

    [输入实例] ->
    """
```

ChatGLM-6B 推理模块用于进行推理，判断句子有无语病、语病的类型，并推理出修改后的正确句子。当文本纠错模型进行初始化时，预先训练好的参数文件会被加载到这个模块中，使得 ChatGLM-6B 模型可以正确回答提示词中指出的任务。这个模块也是整个文本纠错模型的核心模块，它的回答质量和推理所用时间将高度影响到用户的使用体验。

修改意见生成模块根据 ChatGLM-6B 推理模块的返回结果，生成格式化的修改意见。这个修改意见作为 API 的返回结果传递给后端。

3.1.3 后端

本小节主要介绍后端模块的整体结构。后端主要由 Flask 框架和 Python 语言完成开发。

后端主要包括以下 3 个主要功能模块：数据库模块、前端信息接收模块、模型调用模块，如图 3-3 所示。

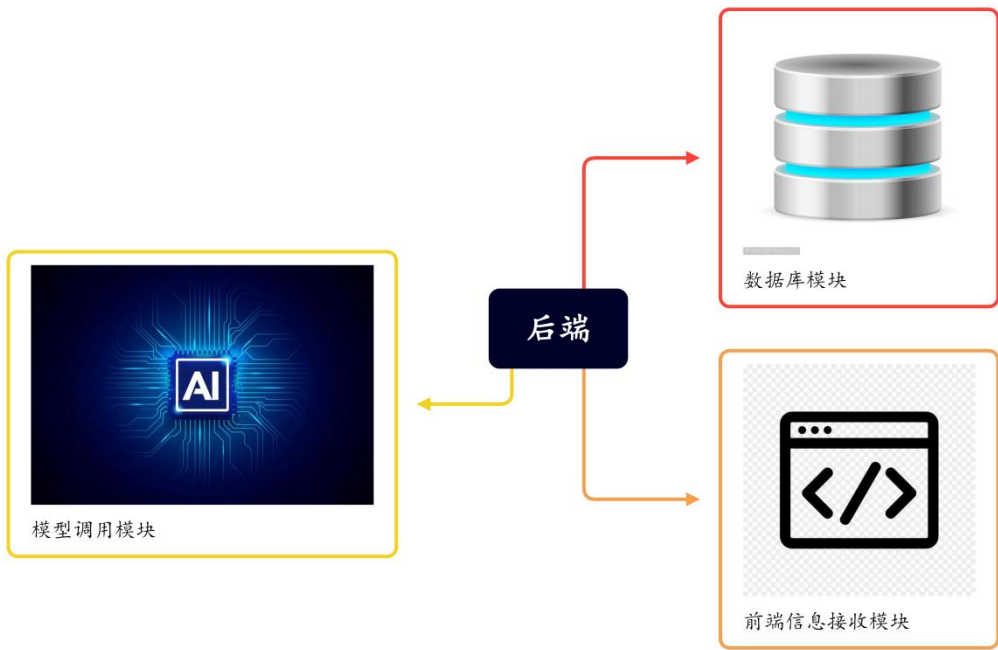


图 3-3 后端系统结构图

数据库模块是整个系统中最核心的部分，负责存储用户注册信息、记录用户登录信息以及保存用户的文件。我们采用 MySQL 数据库实现数据的增、删、改、查操作。在用户进行注册时，数据库模块会接收用户提供的注册信息，并将其存储在数据库中；当用户登录时，数据库模块会验证用户的登录信息并返回相应结果；用户上传文件后，数据库会保存这些文件，并确保它们的安全性、一致性和完整性。

前端接收模块负责处理前端向后端传达的任务。当用户上传文件时，前端接收模块会接收这些文件并对其进行处理。处理完成后，它会通过模型调用模块获得用户文本的纠错结果。

模型调用模块在接到前端接收模块的调用请求之后，调用文本纠错模型并等待模型的结果，模型返回结果后，模型调用模块会将其返回给前端接收模块。

3.1.4 前端

本小节主要介绍前端模块的整体结构。在本系统中，前端主要由 Vue 框架、node.js 运行环境和 TypeScript 语言进行开发。

前端页面主要包括 3 个主要模块：首页模块、帮助模块和用户操作模块，如图 3-4 所示。

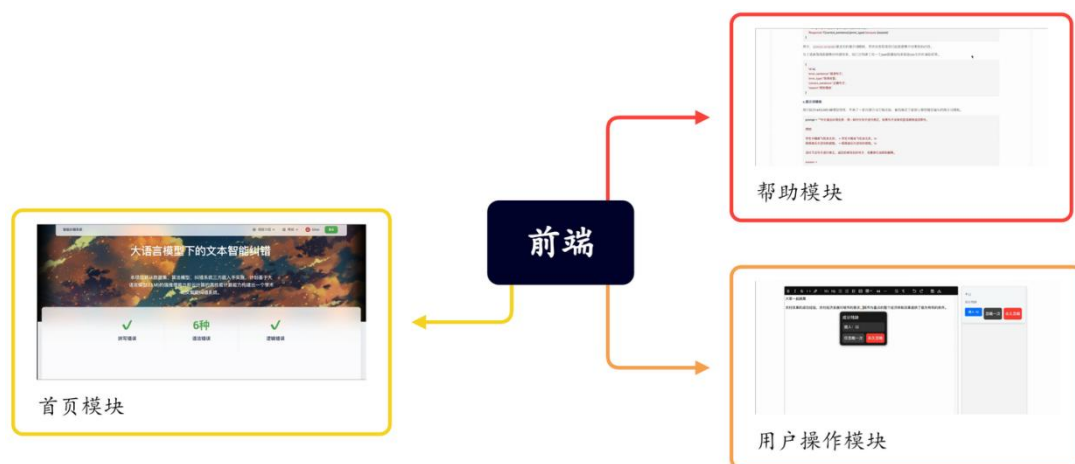


图 3-4 前端系统结构

其中，首页模块包含首页一个页面；帮助模块包含多个项目展示页以及帮助文档页；用户操作模块包含登录页、用户主页和编辑器页面。下面简要介绍这些页面的内容和功能。

● 首页模块

在首页模块中，首页页面简单介绍了系统的主要功能和优势，同时提供了用户进入其它页面的入口按钮。

● 帮助模块

在帮助模块中，各种项目展示页展示了项目的系统结构图、成员名单等相关信息，介绍了项目的开发流程。而帮助文档页为用户提供了整个系统的使用说明来指引用户操作。帮助文档页还为系统管理员提供了部署文档，管理员可根据需求将系统部署在自己的服务器上。

● 用户操作模块

在用户操作模块中，登录页在用户点击登录按钮后通过 `axios` 框架向后端服务器发起登录请求，并在登录成功后自动跳转到首页。当用户处于登录状态下时，可以通过点击导航栏中的链接进入个人界面，用户在该界面中可以上传和管理自己的文档。点击“编辑文档”按钮即可跳转到编辑器界面并展示对应文件。编辑器界面主要有编辑模块和纠错模块，其中编辑模块可以修改文本格式，插入图表等；纠错模块负责向后端发送当前文本，并根据返回的结果进行纠错提示的渲染。

3.2 关键数据结构设计

3.2.1 后端数据结构设计

后端的用户数据存储在数据库中，建表语句如下：

```
CREATE TABLE `userdata` (  
  `id` int(11) NOT NULL AUTO_INCREMENT,  
  `username` varchar(20) NOT NULL,  
  `password` varchar(255) NOT NULL,  
  `role` tinyint(1) NOT NULL,  
  `telephone` varchar(255) NOT NULL,  
  `email` varchar(255) NOT NULL,  
  `address` varchar(255) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `telephone` (`telephone`),  
  UNIQUE KEY `email` (`email`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

3.2.2 前端数据结构设计

前端主要处理用户登录状态相关信息，文件编辑相关信息和纠错结果信息。具体的数据结构参考图3-5。

图3-5中，UserDetail是用户信息类，包括用户名、邮箱等用户相关信息。LoginInfo是登录信息类，存有用户名、用户id和其对应的token。在业务中，前端向后端发起登录请求后，如果成功则会收到一个LoginResponse对象，其中的loginInfo存有需要的token等信息，而userDetail字段为空。如果前端发起查询用户信息请求，则会收到userDetail存在的LoginResponse对象，此时loginInfo字段为空。

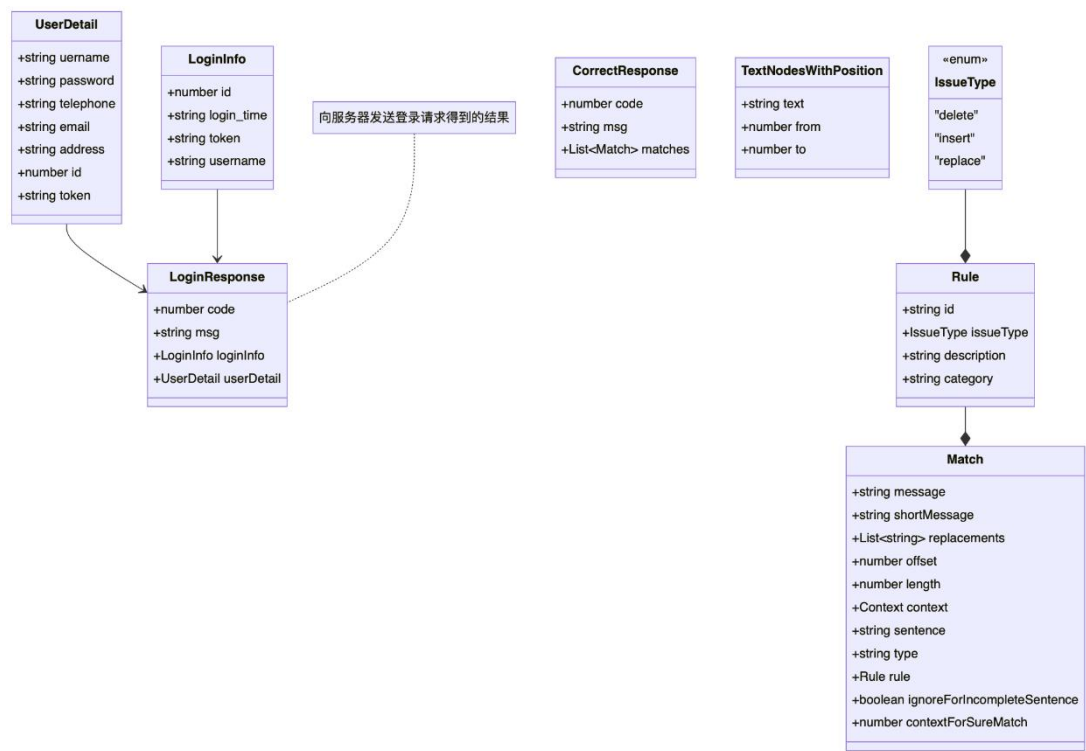


图3-5 前端数据类图

3.2.3 模型端数据结构设计

为了实现模型的纠错功能，我们构建了一个名为LLMCorrector的类。该类共定义了6个内置函数和1个接口。用户只需要实例化LLMCorrector类，便可通过简单地调用接口完成模型的部署以及循环纠错的功能。以下是对该类函数的简单描述。

```
class LLMCorrector():
    def __init__(self,model=None,tokenizer=None):
        """
        初始化函数
        :param model:模型对象
        :param tokenizer:模型分词器对象
        :return: model 对象、tokenizer 对象、对话历史列表
        """

    def __TokenizerInitialize(self,model_params_path):
        """
        初始化模型必需的分词器
        :param model_params_path:模型文件路径，里面也包含了分词器
        :return: 一个重新部署的分词器对象
```

```

"""
def
__ModelInitialize(self,model_params_path,finetune_params_path):
    """
    初始化模型对象
    :param model_params_path:本地下载模型参数地址
    :param finetune_params_path: 微调参数地址，需要精确到文件名
    :return: 重新部署到 GPU 的模型对象
    """
def __get_prompt(self,text):
    """
    构造模型提示词模板
    :param text:待纠错的句子
    :return: 适合模型识别的提示词，跟历史 his 一起使用
    """
def GEC_Correction(self,text):
    """
    接口，模型纠错
    :param text: 输入模型句子。
    :return:修改后返回的编辑序列列表。
    """
def judge(self,response,text,max_unchanged_words=0,threshold=10):
    """
    纠错结果检验，决定该次修改是否保留
    :param response:模型返回结果
    :param text:原句
    :param max_unchanged_words:编辑序列中与原句最大重叠单词数
    :return:bool 变量，表示改句修改是否修改过度，如果修改过度则丢弃
    """

```

剩下的类主要用于纠错功能，存储有错误类型、错误位置、错误长度及修改方法等信息。

3.3 关键算法设计

前后端登录算法的主要流程如图3-6所示。请求报文会携带用户名和密码信

息，响应报文是LoginResponse类的对象，其中的token将会用作之后用户验证的凭证。

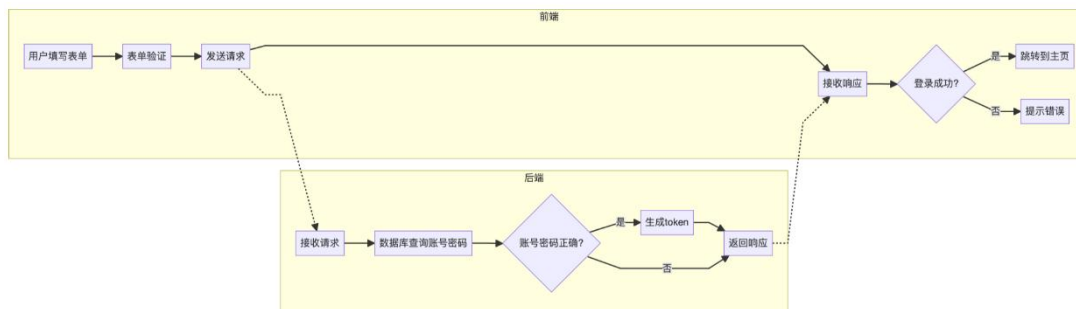


图3-6 登录算法图

文件请求流程如图3-7所示。文件请求报文包含用户名、token、文件名等信息。在后端会首先验证token，验证成功则会读取文件，读取成功后利用pandoc将文档解析为HTML文本，并将文本返回给前端用于后续纠错提示渲染。

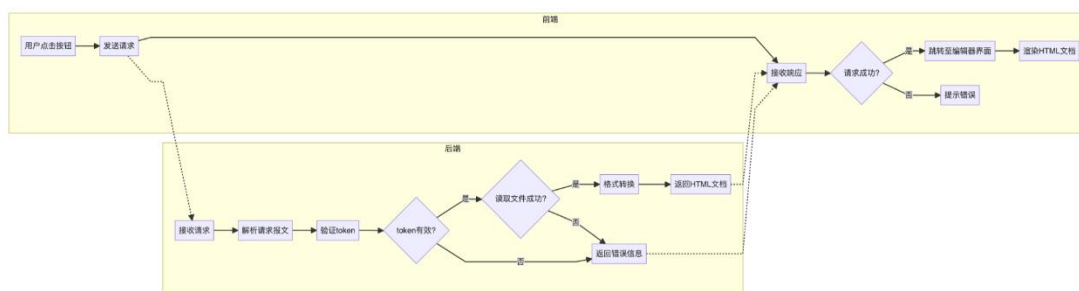


图3-7 文件请求流程图

文件纠错流程如图3-8所示。进入编辑器界面后，前端自动将文本分句并发送给后端。后端调用模型执行纠错，模型向后端返回纠正后的句子，后端再根据模型纠正结果和原句生成编辑序列返回给前端，前端根据编辑序列完成批注渲染，并显示文本修改选项框。

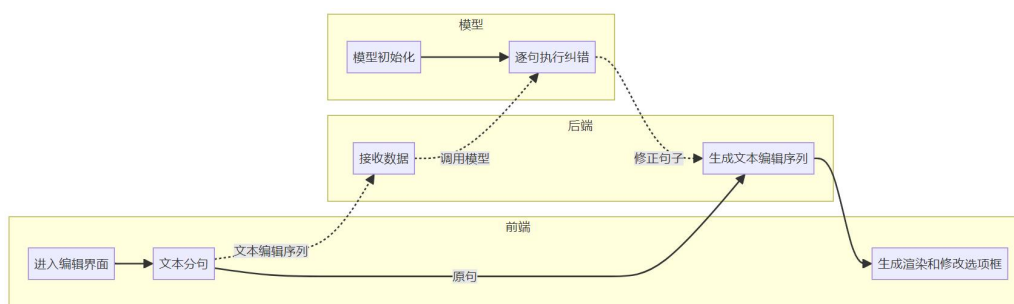


图3-8 文本纠错流程图

模型执行纠错功能的流程如图3-9所示。系统启动时，首先检查GPU是否充足，如果无足够显存则返回错误信息，系统启动失败。模型初始化后处于监听状态，接收待修改序列之后，将原句加入提示词模板并调用模型，针对返回结果与原句进行比对，如果相差过大则认为修改无效，返回原句；否则返回改句。

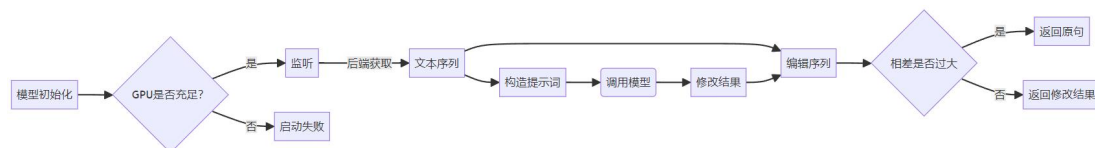


图3-9 模型执行纠错功能流程图

3.4 数据管理说明

3.4.1 用户数据

用户数据采用 MySQL 数据库管理系统管理。

当用户注册且检测注册信息无误后，即同意用户的注册请求，同时分配给用户一个唯一 id 作为主码，将用户的注册信息存入数据库。

3.4.2 用户登录状态

登录状态使用 Redis 数据库维护。Redis（Remote Dictionary Server）是一个使用 ANSI C 编写的开源、支持网络、基于内存、分布式的键值对存储数据库。具体而言，使用 Redis 的键值对存储并跟踪用户的登录状态。使用用户的唯一标识符（用户 ID）作为键，将登录状态作为值进行存储。

登录时，将用户的登录状态存储到 Redis 数据库中。使用一个属性来表示用户的登录状态，将用户 ID 作为键，采用布尔值表示是否已登录。

在需要验证用户登录状态的地方，通过查询 Redis 数据库中存储的键值对来检查用户的登录状态。如果键存在且值为 true 表示用户已经处于登录状态。

当用户退出或会话超时，则更新或删除相应的键值对，将用户状态设置为已退出或过期，然后使用 del 命令删除键，或者设置新的布尔值（如 false）来更新登录状态。

3.4.3 用户文件存储

用户的文档采用文件系统存储，根据用户的 id，将不同用户的文件划分到不

同的目录中。每当用户提交一份文件时，都会为用户对应 id 的目录下储存待纠错的文件。

4 实现与测试

4.1 实现环境与代码管理

4.1.1 系统实现环境

- 硬件设备

本项目在NVIDIA RTX 3090显卡上运行，模型部署和推理占用约12GB显存。

- 虚拟环境

本项目使用conda虚拟环境，虚拟环境所需的库的配置如下所示。

- python >= 3.9
- torch >= 2.0.1
- transformer >= 4.27
- pickle
- jieba
- tqdm
- datasets

- 项目运行

进入项目目录之后，通过执行以下指令即可运行。

- conda activate backend
- python flask_pycorrector.py

此后，使用浏览器登录网址 <http://222.20.95.251:4000/>即可访问项目系统主页。

4.1.2 代码管理

本项目使用Git作为版本控制软件，使用Gitee平台进行在线代码托管，代码仓库地址为<https://gitee.com/GOODGAP/AutoThesisCorrection>。代码版本管理的签入记录如图4-1所示。

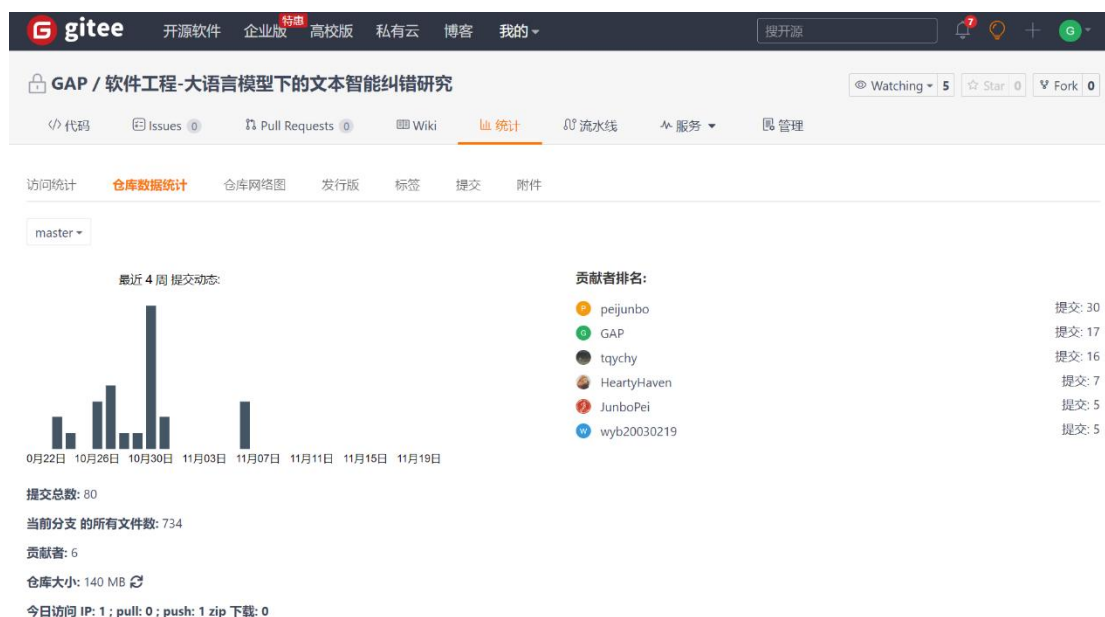


图4-1 Gitee平台代码签入

整个仓库由五位团队成员共同贡献，提交次数分别为：裴俊博同学35次、李嘉鹏同学17次、崔昊阳同学16次、高朗同学7次、王颖彬同学5次。项目仓库概览与仓库网络图分别如图4-2、图4-3所示，由于篇幅限制此处仅节选一部分。

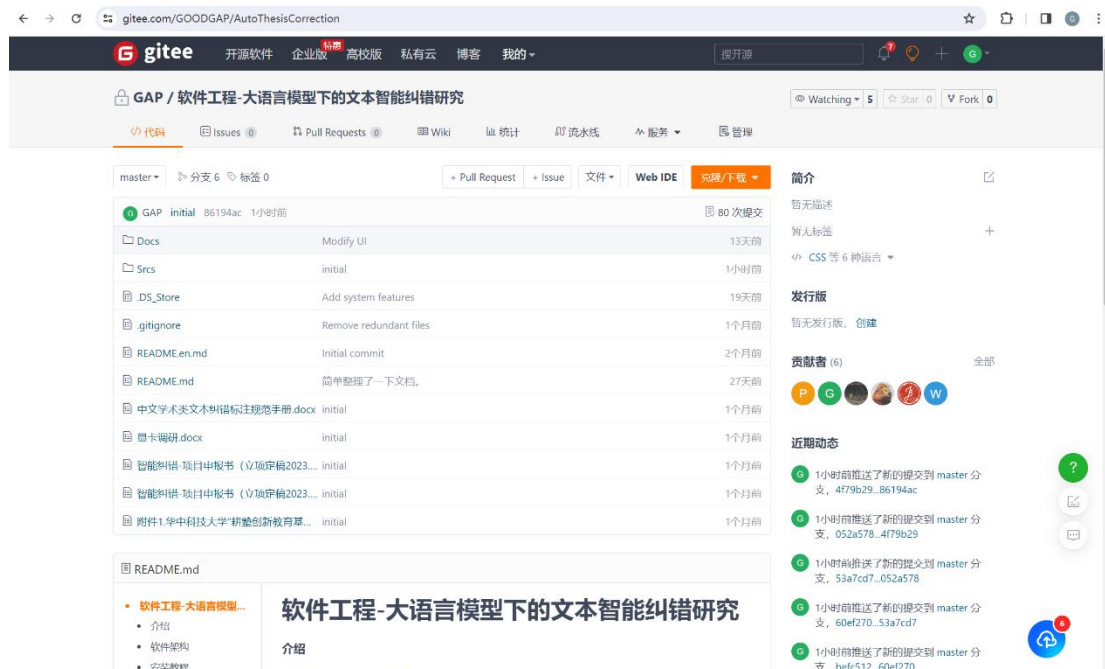


图4-2 项目仓库概览

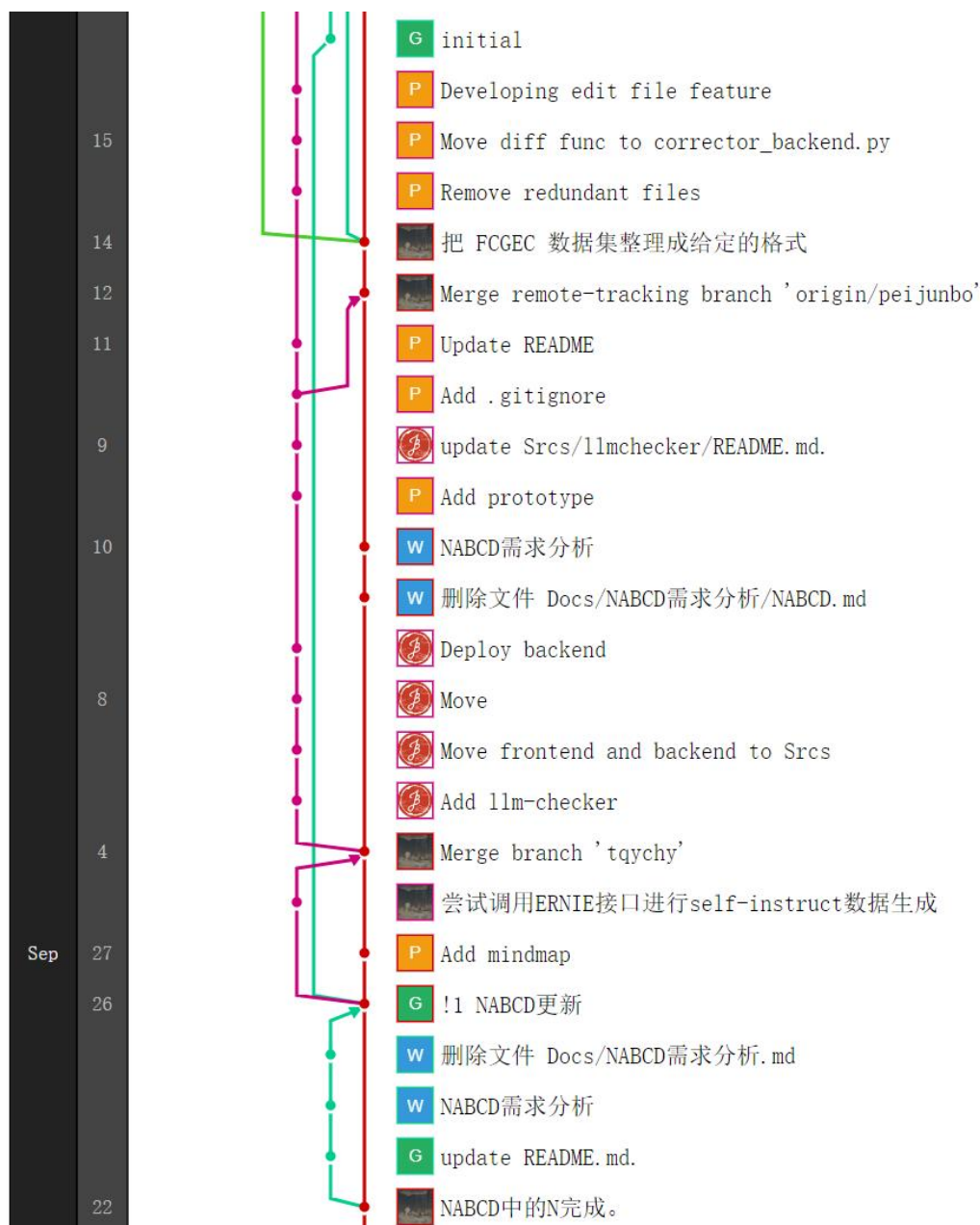


图4-3 仓库网络图（节选）

4.2 关键函数说明

4.2.1 M2 Scorer 评测模块关键函数说明

函数名称：edit_creator

函数功能：对两个文件（source 和 target）中的文本进行编辑距离计算和编辑操作的提取。首先对每一行文本进行预处理，并对其进行分词化（tokenize）。接下来使用 Levenshtein 算法计算距离矩阵和回溯矩阵，并合并两个编辑图，生成并输出该句的编辑序列。

函数名称: tokenizer

函数功能: 将完整的句子按照语法成分拆解为多个独立的短语或词语, 使用中文 jieba 分词器完成语法结构的拆分。

函数名称: m2scorer

函数功能: 通过比对大模型输出的改正后的句子与原始病句 GOLD 标准之间的差异, 计算在 M2 指标下的评分并输出。

整体的处理流程如图 4-4 所示。

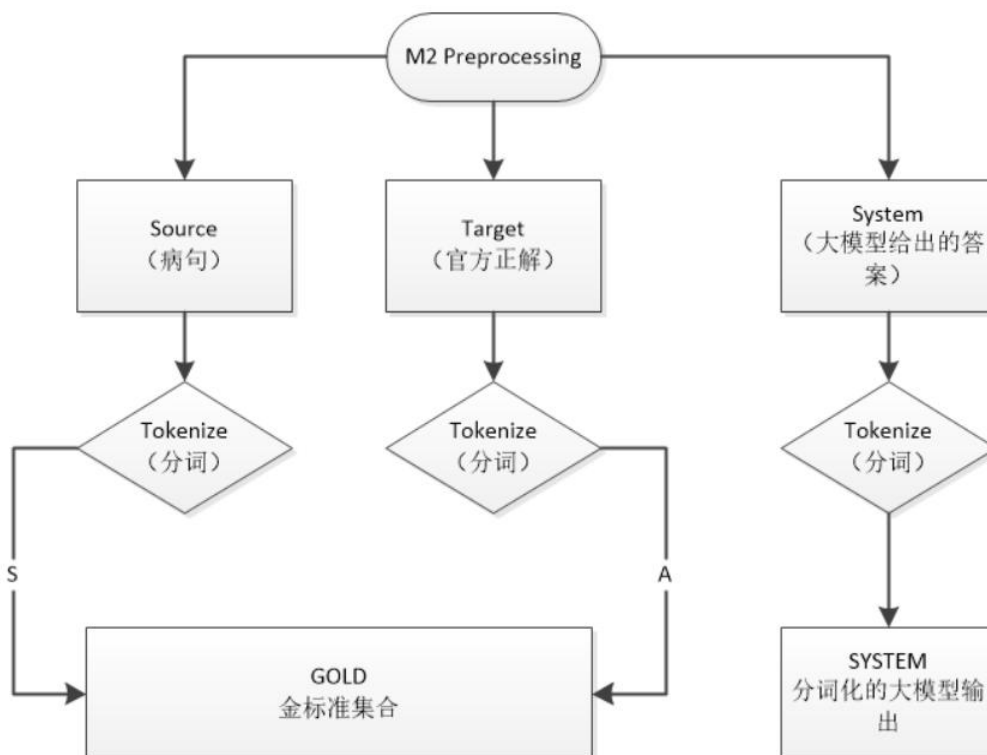


图4-4 M2 scorer模块处理流程

4.2.2 ChatGLM-6B 大语言模型关键函数说明

函数名称: LLMCorrector._TokenizerInitialize

函数功能: 从参数路径中加载预训练模型的专用分词器。

函数名称: LLMCorrector._ModelInitialize

函数功能: 首先检测系统环境中是否有空间充足的 GPU。如果没有则返回

错误信息，否则从参数路径中加载预训练模型的参数文件。

函数名称: LLMCorrector._get_prompt

函数功能: 根据模型接收的待修改文本, 构建促使模型规范修改的提示词(prompt)。

函数名称: LLMCorrector.judge

函数功能: 判断是否保留修改结果。参考了 M2 scorer 的功能实现, 将模型输出的序列与原始序列进行分词(tokenize), 并从编辑序列构造编辑图, 比较二者的交集大小。若交集占比过小则返回原句, 否则返回改句。

函数名称: LLMCorrector.GEC_Correction

函数功能: 首先构建提示词(prompt), 之后传入 ChatGLM-6B 模型, 得到修改后的句子。最后调用 judge 函数, 判断结果是否应该保留, 并返回指定的输出内容。

4.2.3 前端关键对象说明

对象名称: src/router/index.ts router

对象作用: 控制整个系统的页面跳转, 同时为页面跳转进行保护, 例如没有登录的用户不可访问个人信息界面。

对象名称: src/stores/useUserStore.ts

对象作用: 存储全局的应用信息, 同时管理用户登录状态等。

对象名称: src/views/EditorPages/Components/TipTap/TiptapEditor.vue editor

对象作用: 编辑器实例, 通过该实例对象可以在代码中进行编辑器行为的控制, 包括修改内容、提取文本、插入图片、修改格式等。

4.2.4 后端关键函数说明

函数名称: user.get_all_users()

函数功能: 从数据库中检索所有用户的信息。

路线: GET/users

函数名称: user.user_register()

函数功能: 注册新用户。

路线: POST/register

输入: 包含用户名、密码、电话、电子邮件和地址的 JSON 有效负载。

验证: 检查数据库中是否已存在用户名; 验证电话号码格式并检查它是否已注册; 在存储密码之前对密码进行哈希处理。

函数名称: user.user_login()

函数功能: 允许用户登录。

路线: POST/login

输入: 带有用户名和密码的 JSON 有效负载。

验证: 检查用户名是否存在并验证密码; 为登录用户生成一个令牌 (使用 MD5 哈希) 并将其存储在 Redis 中。

函数名称: user.user_update(username)

函数功能: 更新用户信息。

路线: PUT/update/user/int:id

输入: 管理员用户、令牌、新密码、电话、电子邮件和地址。

验证: 检查发出请求的用户是否为管理员; 验证和更新用户信息 (密码、电话、电子邮件、地址)。

函数名称: user.user_delete(username)

函数功能: 删除用户。

路线: POST/delete/user/string:username

输入: 管理员用户和令牌。

验证: 检查发出请求的用户是否为管理员; 从数据库中删除用户。

函数名称: predict.predict()

函数功能：将文本传入模型，调用模型纠错功能。

输入：前端用户输入文本。

函数名称：predict.differ()

函数功能：比较修改后文本和原文本区别，向前端返回错误位置和改正办法。

输入：改前和改后的句子。

返回：标注错误位置即建议更正办法。

4.3 测试计划和测试用例

对于本项目，我们主要采用黑盒测试和面向对象软件测试两种方法测试软件的性能与可靠性。

4.3.1 黑盒测试

黑盒测试是在不考虑内部结构和实现细节的情况下对软件进行测试。主要包括功能测试、用户体验测试和回归测试。

● 功能测试

为了测试软件是否完成预期的功能，我们功能测试的步骤为：①输入各种文本，包括拼写错误、语法错误等；②检查软件是否能准确识别文本中的错误类型；③检查软件是否能给出正确的纠正建议，以及纠正后的文本是否合乎逻辑、语法正确。同时，还会测试软件对于特殊情况的处理，比如输入特定行业术语、缩写等，观察软件是否可以正常检错和纠错。

● 用户体验测试

我们将会收集同学们的使用意见，评估用户界面的友好程度，测试用户操作的直观性和便捷性。同样地，项目组成员也可以模拟用户使用过程，检查软件对于用户输入的响应是否快速，以及提供的建议是否准确、及时。

● 回归测试

在完成前两个测试阶段（功能测试和用户体验测试）后，我们需要修复之前发现的问题，并重新运行相同的测试用例，确保问题已被解决且没有引入新的错误。同时，在后续开发过程中，还需要定期对软件进行测试，以确保新功能的引入不会影响现有功能的稳定性。

综上所述，这些测试环节可以帮助发现智能纠错软件的各种功能和性能问题，提高软件的质量和用户体验。

在对 API 的测试中，我们使用了测试软件 Apifox。该软件可以像语雀、石墨文档一样创建团队文档，拥有相应权限的用户可在其中定义接口和数据模型等，接口定义支持添加 HTTP 请求方法、url 路径、请求体和返回格式等等信息。

4.3.2 面向对象软件测试

面向对象软件测试针对面向对象编程的软件系统进行测试，主要包括单元测试、集成测试、系统测试与安全性测试。对于本项目的“大语言模型下的智能纠错软件系统”，也可以采用面向对象的测试方法。

- **单元测试**

主要针对各个类进行测试，确保类的方法和属性功能正确，这包括测试各个方法的输入输出、边界条件和异常情况处理。同时还需要确保测试用例覆盖了尽可能多的代码路径，包括语句覆盖、分支覆盖等。

- **模块集成测试**

主要是将各个类和模块组合，测试它们之间的交互和集成情况。同时测试类之间的接口和数据传递，确保信息传递正确且无误，各环节都能得到确切的结果。

- **系统测试**

模拟用户使用场景，测试系统的功能是否按照需求分析部分的要求正常工作，这包括测试系统在不同负载下的性能表现，包括响应时间、资源消耗、返回文本修改的准确性等。

- **安全和可靠性测试**

检查系统的安全漏洞，确保用户数据和系统不受攻击。尤其是需要保护用户的隐私，比如在注册时填写的电话、邮箱、密码以及提交的文件信息等。同时，还需要测试系统在长时间运行和异常条件下的稳定性和可靠性。

4.4 结果分析

我们使用上述方法对我们的系统进行了测试，测试结果如下。

- **系统主页**

- 在网络正常的情况下，系统主页可以正常访问。

- 主页无乱码或排版混乱等缺陷。
- 主页上的所有跳转链接均可正常跳转。

该部分测试结果如图 4-5 所示。



图 4-5 系统主页测试（正常显示）

● 用户登录

- 已经注册过的用户可以凭借正确的账号密码成功登录。
- 账号或密码错误时，系统会正常地给予提示，并要求重新输入。
- 非法用户无法通过 SQL 注入、弱密码攻击等常用入侵手段进入系统。
- 未注册的用户可以正常进行注册，当用户填写的注册信息非法时，系统会正常的给予提示。

该部分测试结果如图 4-6~图 4-8 所示。



图 4-6 用户登录界面测试（正常显示）

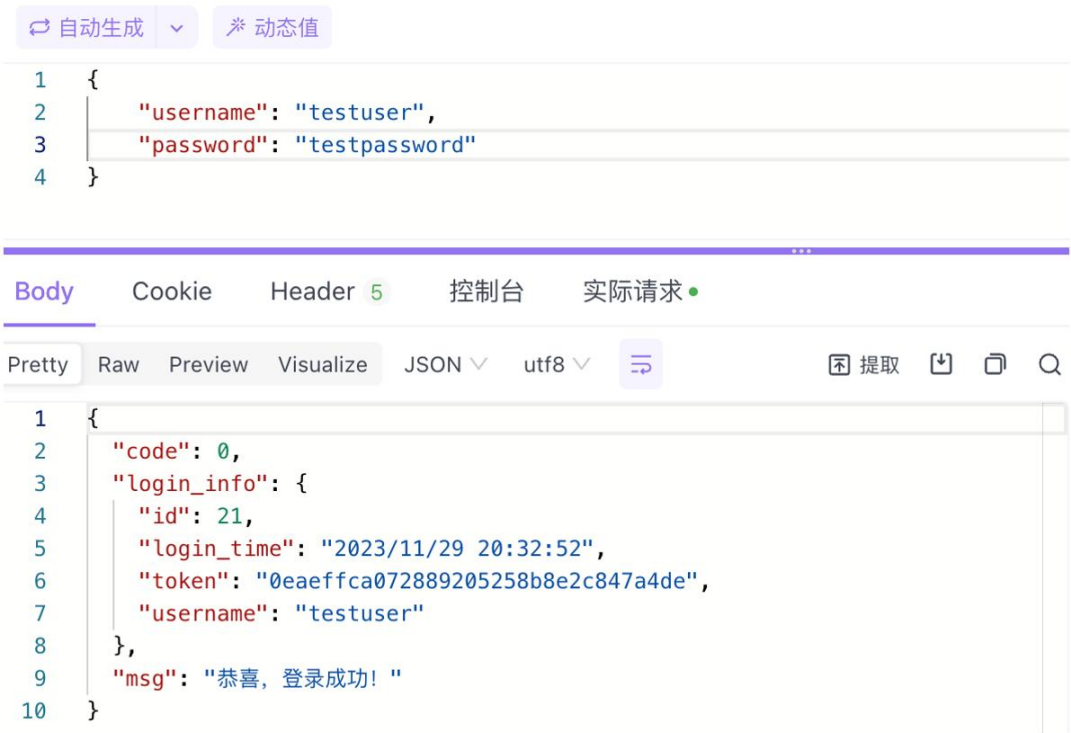


图 4-7 Apifox 测试正确登录



图 4-8 Apifox 测试错误登录

● 文件管理

- 已登录的用户可以正常进入文件上传页面。
- 已登录的用户可以正常上传文件格式受支持的文件。
- 用户无法选择文件格式不受支持的文件。
- 文件上传出现错误时，系统会给予提示，并终止上传过程。
- 用户上传成功的文件可以正常打开，文件格式解析正常，不会出现乱码、格式错误等异常现象。

该部分测试结果如图 4-9、图 4-10 所示。

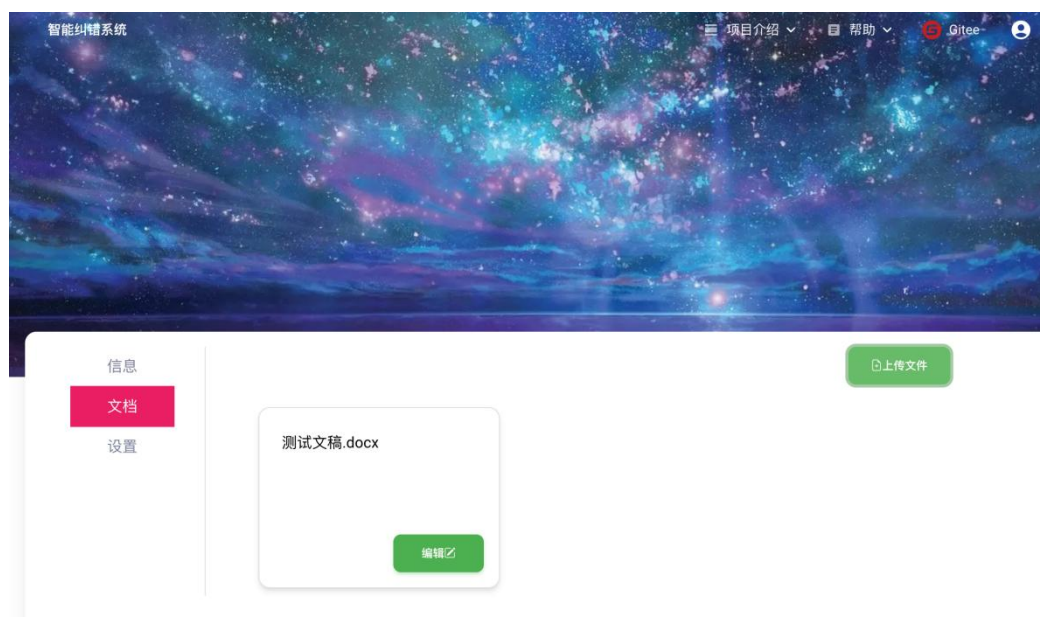


图 4-9 成功获取文件列表

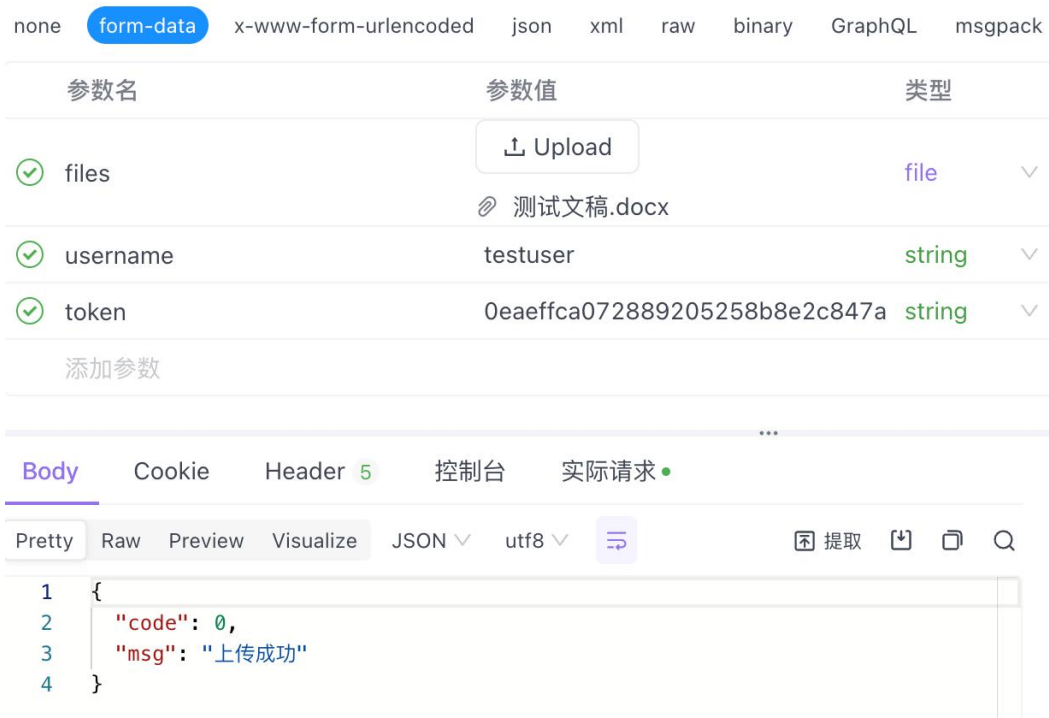


图 4-10 Apifox 测试文件上传成功

- 文本纠错
 - 当用户打开上传的文件时，系统纠错功能运行正常。
 - 系统的错误提示、修改按钮可以正常使用。
 - 系统对各种拼写、语法、语义错误的检出率、改正率较高，且误检率较低。

该部分测试结果如图 4-11 所示。



图 4-11 纠错结果测试

- 在线编辑
 - 在线编辑的编辑器、各种功能按钮等显示正常。
 - 编辑器的功能按钮可以正常使用。
 - 用户编辑后的修改记录可以正常保存至数据库，不会丢失。

综上所述，经过测试可知，本系统的显示内容和各种功能均正常，可以上线运行。

5 总结

5.1 用户反馈

本项目开发完成后，我们将项目系统介绍给一些同学进行试用，使用反馈与照片如下。

(1) 试用案例一

● 使用感受

某 Z 姓同学：“我觉得这个智能文本纠错系统对于我的 word 文档编辑有很大的帮助，可以很方便的帮我修正文本的拼写错误，语法错误，节省了我自己检查纠错的时间。同时，界面也设计的简洁明了，容易上手。美中不足的是有个别错误会检查不出来，可以继续改进算法和模型，进一步优化准确性。同时由于特定行业的术语可能与通用规则有所不同，考虑扩展支持更多的语种和专业领域，以满足不同用户的多样化需求。”（2023 年 11 月 26 日于西十二教学楼）

● 反馈截图

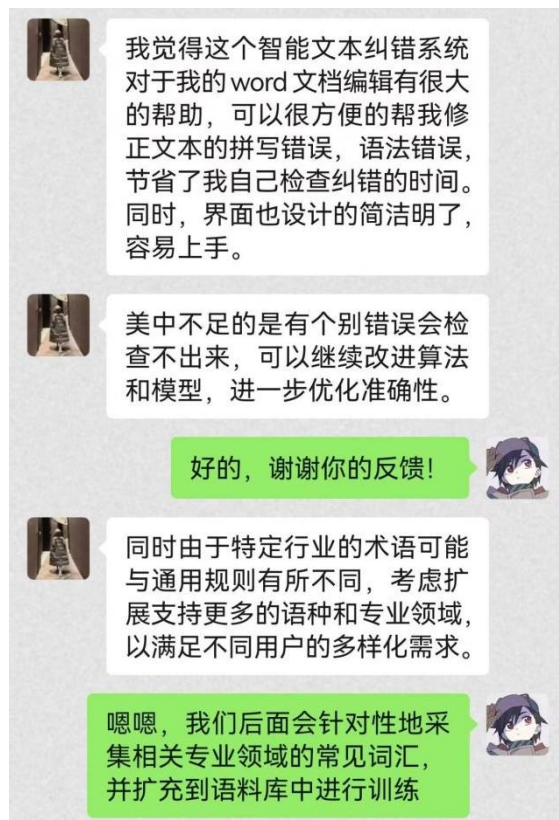


图 5-1 用户反馈一

● 使用照片



图 5-2 用户使用照片一

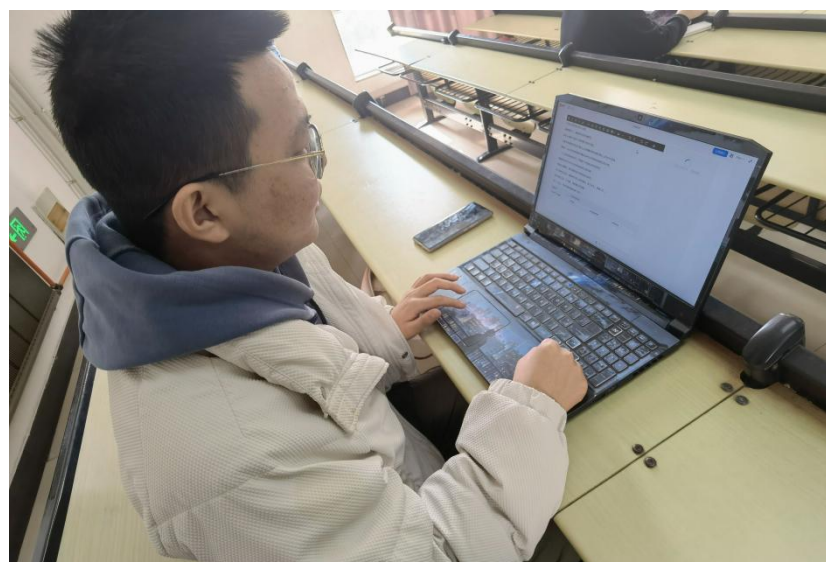


图 5-3 用户使用照片二

(2) 试用案例二

● 使用感受

某 L 姓同学：“这是一个比较稀少但又挺新颖的方向，大模型下的文本智能纠错，不仅帮助学生在论文书写时提供了较大的帮助，而且在利用大模型的背后看到了较大的研究前景。可以考虑加入语言润色等功能，完善一下，做丰富一些。”

（2023 年 11 月 27 日于南一楼）

● 反馈截图

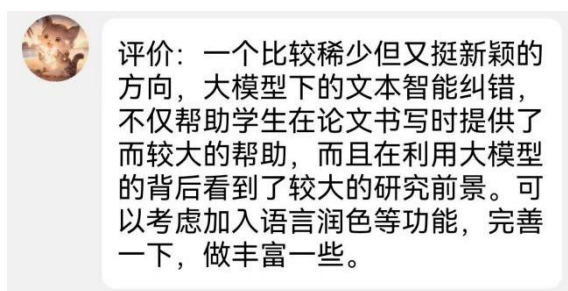


图 5-4 用户反馈二

● 使用照片

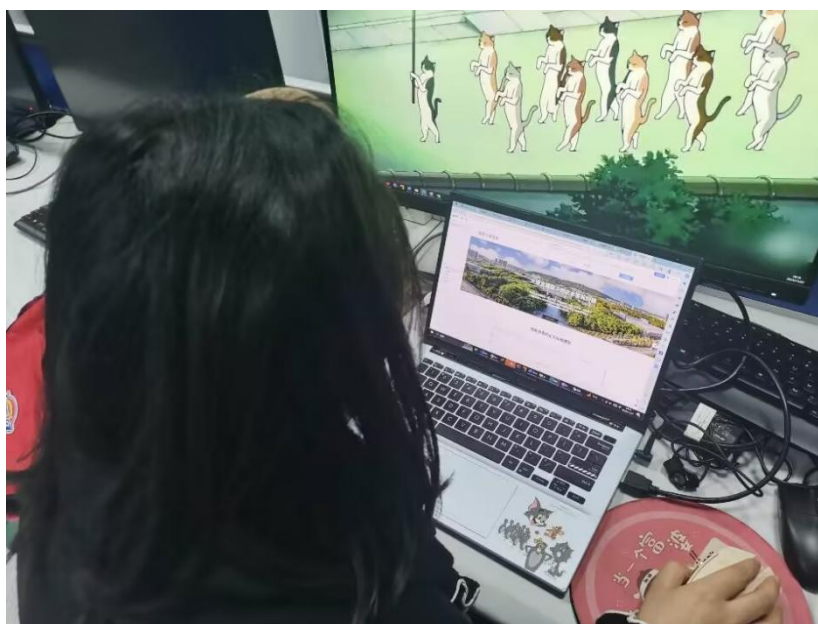


图 5-5 用户使用照片三

5.2 全文总结

(1) 李嘉鹏同学

在软件工程项目的实现中，我主要负责进行项目系统的总体设计，完成模型端的开发、测试与微调，以及对模型效果进行评测，具体工作如下：

①完成了项目背景调研与策划，创建代码仓库并进行代码管理；

②完成了对模型的多轮测试，包括对不规范数据集进行格式修正与检验，以及尝试在不同参数下模型的表现；

③阅读了评测病句修改性能指标的相关论文，并选择了 M2 指标作为检验模型性能的重要依据，跑通了评测代码并结合项目的数据集特征做了修改；

④编写了中文学术类文本纠错标注手册，完成了部分数据集的人工标注；

⑤完成部分软件工程项目报告。

(2) 崔昊阳同学

在软件工程项目的实现中，我主要负责进行大规模数据集的自动生成和模型调试的相关工作。具体工作如下：

- ①完成了现有语法错误数据集的查找和收集；
- ②完成了大规模数据集自动生成方法的查找和比较，实现了大规模标注数据集的自动生成；
- ③参与模型调优、系统测试等任务；
- ④完成部分软件工程项目报告。

(3) 高朗同学

在软件工程项目的实现中，我主要负责模型的部署和调试，以及数据集的构建等工作。具体工作如下：

- ①完成了论文语料的搜集，并合作实现了大规模微调数据集的自动生成；
- ②完成了模型的部署，设计了恰当的提示词模式，维护模型的输出稳定性；
- ③参与模型的多轮评测，负责模型调用、结果清洗；
- ④开展模型微调以及结果测试；
- ⑤完成项目汇报和软件工程项目报告的编写。

(4) 裴俊博同学

在软件工程项目的实现中，我主要负责进行前端界面编写和调试的相关工作。具体工作如下：

- ①完成前端界面的设计；
- ②在开源编辑器框架 TipTap 的基础上编写出了项目需求的编辑器，同时为编辑器添加了对多种富文本格式的支持；
- ③将项目信息等展示在了前端界面中；
- ④与后端对接多个功能，包括用户管理、文件管理和纠错；
- ⑤对前端系统进行测试，以及配合其他同学完成对系统的整体测试。

(5) 王颖彬同学

在软件工程项目的实现中，我主要负责进行后端界面编写和调试的相关工作。
具体工作如下：

- ①完成后端 Flask 架构的搭建；
- ②与前端对接，接收前端文本；
- ③对接大模型，并将模型的纠错结果返回前端；
- ④应用数据库，实现注册用户及文件的记录；
- ⑤对后端系统进行测试，以及配合其他同学完成对系统的整体测试。

5.3 小组会议照片与会议记录

本项目组的五位成员每周定期开展交流汇报会议，确保大家之间的信息相互流通，提高了项目开发的效率。小组会议照片如图 5-6 所示。



图 5-6 小组会议照片

每周会议前，每位成员均需要以 wiki 文档的形式完成上周成果总结报告，并将其上传至语雀平台。我们将会议记录进行了导出和汇总，网页链接为 <https://gitee.com/GOODGAP/AutoThesisCorrection/blob/master/Docs/%E6%99%BA%E8%83%BD%E7%BA%A0%E9%94%99%E5%A4%A7%E5%88%9B%E9%A1%B9%E7%9B%AE%E5%91%A8%E6%8A%A5%E6%B1%87%E6%80%BB.pdf>。

由于篇幅限制，此处仅展示 wiki 文档样例，如图 5-7 所示。



图 5-7 wiki 文档样例

5.4 项目获奖

在两个多月的软件开发中，我们的项目具有了更强的稳定性和鲁棒性，也实现了初期设想的相关功能。基于此，我们将本项目稍作修改并录制了系统展示视频，以“大语言模型下的文本智能纠错系统”为作品参加了 2023 年第五届全球校园人工智能算法精英大赛（计算机学院保研加分 B 类学科竞赛），在算法创新

赛道中获得区域赛一等奖的成绩，如图 5-8 所示，后续即将参加全国总决赛，期待获得更好的成绩！

算法创新赛：其他主题							
序号	团队名称	团队编号	学校名称	团队成员	组别	获奖等级	是否晋级
4	大语言模型下的文本智能纠错系统	bdc23100050	华中科技大学	李嘉鹏、裴俊博、崔昊阳	本科生	一等奖	晋级国赛

图 5-8 项目获奖

6 体会

(1) 李嘉鹏同学

分工：项目策划、模型端构建与微调、模型效果评测

体会：

本学期的软件工程项目来源于之前的一个国家级大创项目，作为项目队长，我从头至尾参与了项目的开发，亲身见证了项目从无到有的发展。在项目开发过程中，我对大语言模型的潜力感到十分惊叹，这类模型的巨大参数量与训练文本量赋予了它远超于此前任何模型的理解与推理能力，在广泛的应用上都有着出色的表现。

同时，我也认识到数据质量对于模型效果的巨大影响。在初期我们采用了目前学术界通用的一些中文病句数据集，但发现它们要么语病不典型，要么数量太少，导致模型微调的效果不佳。后来我们小组分工进行了少量数据集（约 600 条）的人工标注，发现效果就产生了明显提升。

在尝试对模型效果进行评测时，事情总是不会像想象中的那样一帆风顺。首先就是数据集的清洗与预处理很麻烦，因为大模型对输入格式很敏感，其在处理批量 `prompt` 输入时经常返回非预期的结果，导致在处理数据时需要剔除掉乱码等无效数据。其次，由于我们的模型训练的是中文语料，因此传统的 M2 代码中对英语文本进行分词的 `nlk` 分词库实际上效果很差，最后我采用了 `jieba` 分词库才得以解决这个问题。

总体来说，我负责的模型部分要想取得较好的效果，主要就是从数据集和微调两方面入手。通过不断的迭代，我在这一领域也逐渐熟练，学到了很多实践中的 `trick`，也填补了我在大模型领域的知识空白。最后，非常感谢我的队友和指导老师，是大家持续性的帮助才使我们得以走到现在这一步。

(2) 崔昊阳同学

分工：数据集生成和模型调试

体会：

在本系统的实现过程中，我主要负责数据集生成和模型调试。通过本次课程，我学到了很多知识和技能。以下是我的一些体会：

首先，对于参数规模非常大的大语言模型，高质量、大规模的数据集是必要的。数据集的质量高低直接影响了模型的性能和泛化能力。我们尝试了两种方法来构建数据集，一种是从网上爬取论文数据，然后用基于规则的方法来生成具有语法错误的句子（CLG），另一种是利用中文文本生成效果较好的 ERNIE-Bot 4.0 模型来生成错误文本。我们发现后者的效果更好，因为它能够产生更灵活和丰富的语法和语义错误。我们参考了一篇论文，使用了 `self-instruct` 方法来生成大规模、高质量的错误文本。它能够在保持原文本意思的基础上，引入一些合理的语法错误。我们用这个方法生成了大约 4 万条错误文本，作为我们的训练集和测试集。

另外，大语言模型模型的微调是一个非常耗时和复杂的过程，它需要不断地尝试不同的模型结构和参数，以及不同的优化方法和评估指标。经过我、高朗和李嘉鹏同学的合作尝试，我们最终选择了 `ptuning` 微调方法。这种微调方法可以在使用较少计算资源的基础上提高大语言模型在某一特定领域的能力，同时不至于引起模型的灾难性遗忘。

（3）裴俊博同学

分工：前端编写

体会：

对于前端而言，首先一点可能是使用的是一个陌生的框架和编程语言，在此之前我并没有使用过 `Vue` 框架和相应语言。最初，`JavaScript` 的弱类型的特点给我带来了许多困扰，很多时候我不清楚如何编写一份好的代码，这里的好指的是代码风格。经过查找后我了解到 `TypeScript` 语言引入了静态类型检查，在引入 `TypeScript` 后，我明显感觉代码编写起来更加清晰了。

然后在和后端同学交流网络请求时也产生了许多问题。因为计算机网络的课程也在这学期开设，所以对于网络的部分我一开始并不清楚，尤其是 `HTTP` 报文的格式，请求体到底应该放 `JSON` 数据还是表单还是二进制流，这些都影响了我编写的进度。所以期间我较为详细地去了解了相关的知识，也查阅了基本的登录状态处理模式，包括浏览器存储 `token` 或者通过 `cookie` 等方式进行用户验证，以及在令牌过期时如何处理。

同时在学习了软件测试相关课程后,我还了解了在 Vue 框架中如何进行软件测试。在这之前我的测试都是通过运行起来观察系统运行情况的黑盒测试,但学习之后我了解到还有白盒测试等测试方法,这也在一定程度上提升了代码的安全性。

还有就是在这次课程中学会使用了许多帮助管理组织写作项目团队的软件工具。包括使用 mermaid 语言进行各种图的绘制,还有使用语雀进行项目文档的管理以及会议记录,通过 APIFOX 软件进行前后端间 API 的统一协作。

通过这次课程设计,在掌握了一些软件工程方面的相关理论和方法的同时,我的编程能力和团队协作能力也得到了极大的提高。最后,感谢我的老师和队友们的指导和帮助,让我们能够顺利地完成这个项目。

(4) 王颖彬同学

分工: 后端编写

体会:

后端主要采用 Python 语言以及 Flask 架构,Flask 是一个轻量级的 Web 框架,它提供了必要的工具来构建 Web 应用程序,同时保持简单和易于理解。通过智能语言纠错项目,我深刻体会到 flask 架构的诸多优势。

首先,Flask 的简洁性和灵活性是它的一大优势。它不像其他框架那样强制性地定义项目结构,而是允许开发者根据需要自定义。这种灵活性使得在小型项目或原型开发中特别方便,在大型项目中也能发挥作用,因为它允许根据需求组织代码和功能。其次,Flask 的路由系统简单而强大。通过使用装饰器,可以轻松地将 URL 映射到相应的函数。这种直观的映射方式使得管理和维护路由变得非常容易,同时也提供了良好的可读性。Flask 还提供了丰富的扩展库,这些扩展库可以帮助处理各种需求,尤其是表单验证、调用数据库、身份验证等。这些扩展使得开发过程更高效,因为它们提供了预先构建的功能,可以减少开发者的工作量,并且保证了代码的质量和可靠性。

在编写后端代码时,也采用了其他方法提高代码质量和可维护性。比如使用 Blueprints 来组织路由和视图函数。Blueprints 允许将相关的路由和功能组织到单

独的模块中，这有助于代码的可读性和可维护性。其次是利用 Flask 提供的上下文管理，如 request 和 session 对象来处理请求数据和用户会话。

但是在开发过程中也不可避免地遇到了许多问题。首先就是对 flask 架构和 python 语言的不熟悉，还有前后端的接口问题等。幸运的是，老师和队友在开发过程中为我提供了不少帮助，前端开发的同学使用 apifox 统一前后端接口、在前端实现文件转化等，帮我解决了不少问题，也使我能够顺利完成项目。最后，再次感谢我的队友，我们一起前行，共同进步！

(5) 高朗同学

分工：模型构造和调试

体会：

大模型时代，数据是非常重要的，获取数据的能力也是不可或缺的。在项目开始的前期，我参与了数据集的构建，从论文网站上爬取了约 7 万句可用文段，并且与崔昊阳同学合作，将原句生成病句，扩充到 20 万条以上。为了完成这项工作，我利用假期的时间自学网络爬虫，事实证明这项举措是正确的，因为在之后很多情景下都需要用到。

此外，我负责模型的部署和调试，这让我提前熟悉了 GPU 以及 CUDA 开发环境，它们之后被证明是深度学习研究中十分重要的基础能力，为之后的学习和项目开发都带来了一定的便利。

同时，由于模型参数量较少，导致其调用并不如想象中那么顺利，因为其很难理解用户明确的指向性信息。所以在纠错的时候，很容易返回一些无关的信息（输出自己思考的过程，以及不稳定的结构化内容等），为此，我也付出了较大的努力，尝试从提示词构建上入手，提升模型输出的稳定性。最后，我发现使用 few-shot 提示词技术，加以构造对话历史，能够让模型更好地识别我的指令信息。最终成功实现了输入病句，输出改句，而不掺杂任何无关信息的效果。

我也参与了模型的微调工作。我们首先使用 20 万条自动生成的病句作为数据集对模型开展了 P-Tuning 微调。最终，模型能够很轻易地分辨出这 20 万句中出现的病句，但随之而来的是基础对话能力的显著下滑，以及灾难性遗忘。而且，

模型并不能将所学知识进行迁移。我们认为这是数据集质量过低引发的问题。因此，我们参考相关研究，决定人工构造更高质量的语料。

以上这些工作，为我的科研和项目开发奠定了很好的基础，也改变了我的思维方式，熟悉 CPU 和 GPU 开发环境的种种区别，以及如何有效地利用 GPU 进行算法优化，利用并行减少时间开销等等。最后，感谢同学与老师的帮助和努力，使这个项目得以成功落地。

参考文献

- [1] Zeng, Aohan.etc (2022). GLM-130B: An Open Bilingual Pre-trained Model.
- [2] Wang, Yizhong.etc (2022). Self-Instruct: Aligning Language Model with Self Generated Instructions. 10.48550/arXiv.2212.10560.
- [3] Ma Shirong.etc (2022). Linguistic Rules-Based Corpus Generation for Native Chinese Grammatical Error Correction. arXiv.2210.10442.
- [4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- [5] Chollampatt, S., Ng, H. T., & Luo, Z. (2018). Ensembling and knowledge distilling of large sequence taggers for grammatical error correction. arXiv preprint arXiv:1810.06799.3. Zhang, Y., & Sun, M. (2018). Interpretability for language learners. arXiv preprint arXiv:1810.03675.
- [6] 王辰成, 陈嘉琪, 张瑞琦. 基于 Transformer 增强架构的中文语法纠错方法[J]. 计算机学报, 2020, 43(6): 106-118.
- [7] Chollampatt, S., Ng, H. T., & Luo, Z. (2019). Adjusting the precision-recall trade-off with align-and-predict decoding for grammatical error correction. arXiv preprint arXiv:1901.09528.
- [8] Chollampatt, S., Ng, H. T., & Luo, Z. (2019). Reusing a multi-lingual setup to bootstrap a grammar checker for a very low resource language without data. arXiv preprint arXiv:1901.09528.
- [9] Liu, X., He, Y., & Sun, M. (2019). “Is Whole Word Masking Always Better for Chinese BERT?”: Probing on Chinese Grammatical Error Correction.
- [10] Liang, C., Liang, X., Sun, M., & Huang, J.-T. (2019). MDCSpell: A Multi-task Detector-Corrector Framework for Chinese Spelling Correction

附录

我们在此给出模型端、前端、后端和 M2 scorer 的部分关键代码。

附录 A 模型端关键代码

```
class LLMCorrector():
    def __init__(self,model=None,tokenizer=None):
        """
        :param model:模型对象
        :param tokenizer: 模型分词器对象
        """
        pynvml.nvmlInit()
        self.model=model if model!=None else
self.__ModelInitialize('/home/grammar/Paper_Corrector/backend_interact/mymodel','/h
ome/grammar/Paper_Corrector/backend_interact/CGEC-chatglm2-6b-pt-128-2e-2/checkpoin
t-3000/pytorch_model.bin')#如果没有指定模型，此函数重新部署模型，在这里修改参数。
        self.tokenizer=tokenizer if tokenizer!=None else
self.__TokenizerInitialize('/home/grammar/Paper_Corrector/backend_interact/mymodel')
#如果没有加载分词器，则重新加载分词器，路径在此处修改。
        self.his=[("因为污染也很高，我们要回收垃圾。为什么?塑料、电池、垃圾是我们做的东西。
-> ","因为污染很严重，所以我们要回收垃圾。为什么?因为塑料、电池等垃圾是我们人类造出来的东西。
"),
                ("最近吸烟者率是越来越多，而且吸烟者的年龄层是越来越少。这些问题是现在最重要
的事。 -> ","最近吸烟率是越来越高，而且吸烟者的年龄层是越来越低。这些问题是现在最重要的事。
"),
                ("不仅他学习某种习惯，他们性格也开始养成。 -> ","他们不仅开始学习某种习惯，
他们的性格也开始形成。"),
                ("这世界上是很残酷的。 -> ","这世界是很残酷的。"),
                ("每一刻都用得无可挑剔。 -> ","每一刻都用得无可挑剔。")]#人为构造的模型的
历史记录，不用更改

    def __TokenizerInitialize(self,model_params_path):
        """
        :param model_params_path:模型文件路径，里面也包含了分词器
        :return: 一个重新部署的分词器对象
        """
        # 载入 Tokenizer
        tokenizer = AutoTokenizer.from_pretrained(model_params_path,
trust_remote_code=True)
        return tokenizer
```

```
def __ModelInitialize(self,model_params_path,finetune_params_path):
    """
    :param model_params_path:本地下载模型参数地址
    :param finetune_params_path: 微调参数地址，需要精确到文件名
    :return: 重新部署到 GPU 的模型对象
    """

    # 检查是否有 GPU 以及 GPU 是否有足够内存加载模型。
    available_gpu_exists=False
    if torch.cuda.is_available():
        device_count=torch.cuda.device_count()
        for i in range(device_count):# 这里遍历所有的 GPU 找能用的
            handle = pynvml.nvmlDeviceGetHandleByIndex(i)
            meminfo = pynvml.nvmlDeviceGetMemoryInfo(handle)#这些都是模板
            if(meminfo.free/1024**3>=16):    #chatglm-6b 占用显存 11-12G，这里找至少
13G 可用显存的 GPU，要是资源实在紧张可以把这个改成'>=12'。
                print(f'Device {i} is available!')
                device = torch.device(f"cuda:{i}") # 获取当前 GPU 设备#这里用完之后
记得改回 i

                available_gpu_exists=True
                break
        if available_gpu_exists==False:
            raise MemoryError('No GPU available to load model.Please check whether
the model is already loaded.')
        else:
            raise ModuleNotFoundError('No GPU found on this device,please check if you
are running this program on GPU.')

    # 加载 chatglm2-6b 模型
    config = AutoConfig.from_pretrained(model_params_path, trust_remote_code=True,
pre_seq_len=128)
    #config 是用来配置一部分模型参数的，不用改动。
    model = AutoModel.from_pretrained(model_params_path, trust_remote_code=True)
    #加载微调参数文件

    #-----#
    prefix_state_dict = torch.load(
        finetune_params_path)#根据实际情况修改这一行为微调参数文件路径，要精确到.bin
文件
    new_prefix_state_dict = {}
    for k, v in prefix_state_dict.items():
        if k.startswith("transformer.prefix_encoder."):

```



```
new_prefix_state_dict[k[len("transformer.prefix_encoder.") :]] = v
model.transformer.prefix_encoder.load_state_dict(new_prefix_state_dict)
#-----#

# 量化操作，减少模型的大小和计算量，不影响性能。
model = model.quantize(4) # 参数映射为 8 位整数，4bit 的量化精度，也可以设定为 8.
model = model.half().cuda()
model.transformer.prefix_encoder.float()
# 设置为评估模式，推理速度更快。

model = model.to(device) # 将模型移动到当前 GPU 设备上
model = model.eval()

print('Successfully built model ChatGLM2-6B!')
return model

def __get_prompt(self, text):
    """
    :param text: 待纠错的句子
    :return: 适合模型识别的提示词，跟历史 his 一起使用
    """
    prompt = """中文语法纠错任务：将一段中文句子进行修正，如果句子没有明显语病则返回原
句。

下面是一些范例：

学生大概做飞机去北京。 -> 学生大概坐飞机去北京。 \n
我觉得他很大胆，他不外国。他觉得如果要改变中国，所以就要从中国做的研究。 -> 我
觉得他很大胆，他不外国。他觉得如果要改变中国，就要在中国做研究。 \n
吸烟者反对这样的措施。 -> 吸烟者反对这样的措施。 \n

请对下述句子进行修正。返回你修改后的句子，无需其它说明和解释。

xxxxxx ->

"""
    return prompt.replace('xxxxxx', text)

def GEC_Classification(self):
    """
    :return: 暂时还没写。
    """
```

```

pass

def GEC_Correction(self, text):
    """
    :param text: 输入模型的句子。
    :return: 修改后返回的编辑序列列表。
    """
    prompt = self.__get_prompt(text)
    response, _ = self.model.chat(self.tokenizer, prompt, history=self.his)
    # edits=self.__Resp_to_Edit(response, text)
    response=self.judge(response, text, threshold=4)
    return response

def judge(self, response, text, max_unchanged_words=0, threshold=10):
    """
    :param response
    :param text
    :param max_unchanged_words:
    """
    beta = 0.5
    ignore_whitespace_casing = False # - Ignore edits that only affect whitespace
    and casing. Default no.
    verbose = False # - print verbose output"
    very_verbose = False # - print lots of verbose output
    if(response==text):
        return response
    text = text.strip()
    response = response.strip()
    text_tok = ' '.join(jieba.cut(text, cut_all=False)).split()
    response_tok = ' '.join(jieba.cut(response, cut_all=False)).split()
    lmatrix1, backpointers1 = levenshtein_matrix(response_tok, text_tok, 1, 1, 1)
    lmatrix2, backpointers2 = levenshtein_matrix(response_tok, text_tok, 1, 1, 2)
    V1, E1, dist1, edits1 = edit_graph(lmatrix1, backpointers1)
    V2, E2, dist2, edits2 = edit_graph(lmatrix2, backpointers2)
    V, E, dist, edits = merge_graph(V1, V2, E1, E2, dist1, dist2, edits1, edits2)
    V, E, dist, edits = transitive_arcs(V, E, dist, edits, max_unchanged_words,
    very_verbose)
    # Find the shortest path with an empty gold set
    gold = []
    localdist = levenshtein.set_weights(E, dist, edits, gold, verbose, very_verbose)
    editSeq = levenshtein.best_edit_seq_bf(V, E, localdist, edits, very_verbose)
    if ignore_whitespace_casing:
        editSeq = filter(lambda x: not equals_ignore_whitespace_casing(x[2], x[3]),
    editSeq)

```

```
if abs(editSeq[0][1] - editSeq[0][0]) >= threshold:  
    return text  
return response
```

附录 B 前端关键代码

前端主代码，负责启动前端 app

```
import { createApp } from "vue";
import { createPinia } from "pinia";
import App from "./App.vue";
import router from "./router";
// Nucleo Icons
import "./assets/css/nucleo-icons.css";
import "./assets/css/nucleo-svg.css";
import "bootstrap-icons/font/bootstrap-icons.css"
import "bootstrap/dist/css/bootstrap.css";
import materialKit from "./material-kit";
import "highlight.js/styles/github-dark.css";
import hljs from "highlight.js/lib/core";
import hljsVuePlugin from "@highlightjs/vue-plugin";
import bash from "highlight.js/lib/languages/bash";
import sql from "highlight.js/lib/languages/sql";
import shell from "highlight.js/lib/languages/shell";
hljs.registerLanguage("bash", bash);
hljs.registerLanguage("sql", sql);
hljs.registerLanguage("shell", shell);
const app = createApp(App);
app.use(createPinia());
app.use(router);
app.use(hljsVuePlugin);
app.use(materialKit);
app.mount("#app");
```

创建编辑器实例

```
const editor = useEditor({
  extensions: [
    StarterKit,
    Highlight,
    CodeBlock,
    TaskList,
    TaskItem.configure({
      nested: true
    }),
    LanguageTool.configure({
      automaticMode: true,
      documentId: '1',
      apiUrl: 'http://222.20.97.104:8081/correct/',
    }),
    Table as any,
    TableCell,
    TableHeader,
    TableRow,
    Image.configure({
      inline: true,
      allowBase64: true,
    })
  ],
  onUpdate({ editor }) {
    setTimeout(() => updateMatch(editor as any))
  },
  onSelectionUpdate({ editor }) {
    setTimeout(() => updateMatch(editor as any))
  },
})
```

```
onTransaction({ transaction: tr }) {  
  if (tr.getMeta(LanguageToolHelpingWords.LoadingTransactionName)) {  
    console.log('loading')  
    loading.value = true  
  }  
  else {  
    console.log('not loading')  
    loading.value = false  
  }  
},  
})
```

创建用户信息存储实例

```
export const useUserStore = defineStore("userDetail", () => {  
  const mUserDetail = ref<UserDetail>(getSessionUserDetail() ?? emptyUserDetail());  
  const userDetail = computed<UserDetail>({  
    get() {  
      return mUserDetail.value;  
    },  
    set(newUserDetail: UserDetail) {  
      console.log("set userDetail")  
      setSessionUserDetail(newUserDetail);  
      mUserDetail.value = newUserDetail  
    }  
  })  
  const isLogin = computed<boolean>(() => {  
    return userDetail.value.token !== undefined && userDetail.value.token.length > 0  
  })  
  return {  

```

```
    userDetails,  
    isLogin,  
  };  
});
```


附录 C 后端关键代码

```
def diff(a, b):
    """
    比较两个字符串的差异，输出需要变换的操作序列
    :param a: 字符串 a      :param b: 字符串 b
    """
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j - 1])
    # 记录操作序列
    res = []
    """
    Operation:
    0, not change    1. add    2. delete    3. modify
    """
    i, j = m, n
    while i != 0 or j != 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            res.append([0, ""])
            i -= 1
            j -= 1
        else:
            if i > 0 and dp[i][j] == dp[i - 1][j] + 1:
                res.append([2, a[i - 1]])
                i -= 1
            elif j > 0 and dp[i][j] == dp[i][j - 1] + 1:
                res.append([1, b[j - 1]])
                j -= 1
            else:
                res.append([3, b[j - 1]])
                i -= 1
                j -= 1
    res.reverse()
    return res
```

```

if __name__ == "__main__":
    a = "I ad a boy"
    b = "I dont am a boy"
    print(diff(a, b))

#coding utf-8
parser = argparse.ArgumentParser()
parser.add_argument('-m', '--model_path', default=r'Simple Flask
Backend\Models\exps\seq2seq_lang8+hsk')
parser.add_argument('-i', '--input_path', default=r'Simple Flask
Backend\Models\test.txt')
parser.add_argument('-o', '--output_path', default=r'Simple Flask
Backend\Models\res.txt')
parser.add_argument('-b', '--batch_size', default=50)
args = parser.parse_args()
cc = OpenCC("t2s")
tokenizer=BertTokenizer.from_pretrained(args.model_path)
model=BartForConditionalGeneration.from_pretrained(args.model_path)
model.eval()
model.half()
model.cuda()

def split_sentence(document: str, flag: str = "all", limit: int = 510):
    """
    Args:
        document:
        flag: Type:str, "all" 中英文标点分句, "zh" 中文标点分句, "en" 英文标点分句
        limit: 默认单句最大长度为 510 个字符

    Returns: Type:list
    """
    sent_list = []
    try:
        if flag == "zh":
            document = re.sub('(P<quotation_mark>([。？！])(?!["'"\'])))',
r'\g<quotation_mark>\n', document) # 单字符断句符
            document = re.sub('(P<quotation_mark>([。？！])["'"\']')',
r'\g<quotation_mark>\n', document) # 特殊引号

        elif flag == "en":
            document = re.sub('(P<quotation_mark>([.?!])(?!["'"\'])))',
r'\g<quotation_mark>\n', document) # 英文单字符断句符

```

```

        document = re.sub('(P<quotation_mark>([?!."'\'])))',
r'\g<quotation_mark>\n', document) # 特殊引号

    else:
        document = re.sub('(P<quotation_mark>([。? ! ...?!](?!["'"\'])))',
r'\g<quotation_mark>\n', document) # 单字符断句符
        document = re.sub('(P<quotation_mark>(([,。? ! .!?]|...{1,2})[""'\'])))',
r'\g<quotation_mark>\n', document) # 特殊引号

    sent_list_ori = document.splitlines()
    for sent in sent_list_ori:
        sent = sent.strip()
        if not sent:
            continue
        else:
            while len(sent) > limit:
                temp = sent[0:limit]
                sent_list.append(temp)
                sent = sent[limit:]
            sent_list.append(sent)
    except:
        sent_list.clear()
        sent_list.append(document)
    return sent_list

def run_model(sents):
    num_ret_seqs = 1
    beam = 5
    inp_max_len = 100
    batch = [tokenizer(s, return_tensors='pt', padding='max_length',
max_length=inp_max_len) for s in sents]
    oidx2bidx = {} #original index to final batch index
    final_batch = []
    for oidx, elm in enumerate(batch):
        if elm['input_ids'].size(1) <= inp_max_len:
            oidx2bidx[oidx] = len(final_batch)
            final_batch.append(elm)
    batch = {key: torch.cat([elm[key] for elm in final_batch], dim=0) for key in
final_batch[0]}
    with torch.no_grad():
        generated_ids = model.generate(batch['input_ids'].cuda(),
attention_mask=batch['attention_mask'].cuda(),

```

```

num_beams=beam, num_return_sequences=num_ret_seqs,
max_length=inp_max_len)
_out = tokenizer.batch_decode(generated_ids.detach().cpu(),
skip_special_tokens=True)
outs = []
for i in range(0, len(_out), num_ret_seqs):
    outs.append(_out[i:i+num_ret_seqs])
final_outs = [[sents[oidx]] if oidx not in oidx2bidx else outs[oidx2bidx[oidx]] for
oidx in range(len(sents))]
return final_outs

def predict():
    sents = [l.strip() for l in open(args.input_path,encoding='utf-8'
)] # 分句
    subsents = []
    s_map = []
    for i, sent in enumerate(sents): # 将篇章划分为子句，分句预测再合并
        subsent_list = split_sentence(sent, "zh")
        s_map.extend([i for _ in range(len(subsent_list))])
        subsents.extend(subsent_list)
    assert len(subsents) == len(s_map)
    b_size = args.batch_size
    outs = []
    for j in tqdm(range(0, len(subsents), b_size)):
        sents_batch = subsents[j:j+b_size]
        outs_batch = run_model(sents_batch)
        for sent, preds in zip(sents_batch, outs_batch):
            outs.append({'src': sent, 'preds': preds})
    results = ["" for _ in range(len(sents))]
    with open(args.output_path, 'w') as outf:
        for i, out in enumerate(outs):
            results[s_map[i]] += cc.convert(out['preds'])[0])
        for res in results:
            outf.write(res + "\n")

predict()

```

附录 D M2 Scorer 关键代码

```
# m2scorer 代码
import re
import sys

class DummyTokenizer(object):

    def tokenize(self, text):
        return text.split()

class PTBTokenizer(object):

    def __init__(self, language="en"):
        self.language = language
        self.nonbreaking_prefixes = {}
        self.nonbreaking_prefixes_numeric = {}
        self.nonbreaking_prefixes["en"] = ''' A B C D E F G H I J K L M N O P Q R S T
U V W X Y Z
Adj Adm Adv Asst Bart Bldg Brig Bros Capt Cmdr Col Comdr Con Corp Cpl DR Dr
Drs Ens
Gen Gov Hon Hr Hosp Insp Lt MM MR MRS MS Maj Messrs Mlle Mme Mr Mrs Ms Msgr
Op Ord
Pfc Ph Prof Pvt Rep Reps Res Rev Rt Sen Sens Sfc Sgt Sr St Supt Surg
v vs i.e rev e.g Nos Nr'''.split()
        self.nonbreaking_prefixes_numeric["en"] = '''No Art pp'''.split()
        self.special_chars = re.compile(r"([\w\s\.\'\`\,\-\\"|\/])",
flags=re.UNICODE)

    def tokenize(self, text, ptb=False):
        text = text.strip()
        text = " " + text + " "

        text = re.sub(self.special_chars, r' \1 ', text)
        text = re.sub(r";", r' ; ', text)
        text = re.sub(r":", r' : ', text)
        text = re.sub(r"\|", r' -PIPE- ', text)
        text = re.sub(r"(\S)/(\S)", r'\1 / \2', text)

        # 分词化
        if ptb:
            text = re.sub(r"\(", r' -LRB- ', text)
            text = re.sub(r"\)", r' -RRB- ', text)
            text = re.sub(r"\[", r' -LSB- ', text)
            text = re.sub(r"\]", r' -RSB- ', text)
            text = re.sub(r"\{", r' -LCB- ', text)
```

```

text = re.sub(r"\}", r' -RCB- ', text)

text = re.sub(r"\\"s*$", r" ' ' ", text)
text = re.sub(r"^\\s*\"", r' `` ', text)
text = re.sub(r"(\S)\\s", r"\1 ' ' ", text)
text = re.sub(r"\\s\"(\S)", r" `` \1", text)
text = re.sub(r"(\S)\\s", r"\1 ' ' ", text)
text = re.sub(r"\"(\S)", r" `` \1", text)
text = re.sub(r"'\\"s*$", r" ' ' ", text)
text = re.sub(r"^\\s*'\"", r" ` ", text)
text = re.sub(r"(\S)'\\"s", r"\1 ' ' ", text)
text = re.sub(r"\\s'(\S)", r" ` \1", text)

text = re.sub(r"'11", r" -CONTRACT-11", text)
text = re.sub(r"'re", r" -CONTRACT-re", text)
text = re.sub(r"'ve", r" -CONTRACT-ve", text)
text = re.sub(r"'n't", r" n-CONTRACT-t", text)
text = re.sub(r"'LL", r" -CONTRACT-LL", text)
text = re.sub(r"'RE", r" -CONTRACT-RE", text)
text = re.sub(r"'VE", r" -CONTRACT-VE", text)
text = re.sub(r"'N'T", r" N-CONTRACT-T", text)
text = re.sub(r"cannot", r"can not", text)
text = re.sub(r"Cannot", r"Can not", text)

text = re.sub(r"\.([\.\.]+)", r" DOTMULTI\1", text)
while re.search("DOTMULTI\.", text):
    text = re.sub(r"DOTMULTI\.([\.\.])", r"DOTDOTMULTI \1", text)
    text = re.sub(r"DOTMULTI\.", r"DOTDOTMULTI", text)

text = re.sub(r"\-([\.\.]+)", r" DASHMULTI\1", text)
while re.search("DASHMULTI\-", text):
    text = re.sub(r"DASHMULTI\-(^[\.\.])", r"DASHDASHMULTI \1", text)
    text = re.sub(r"DASHMULTI\-", r"DASHDASHMULTI", text)

text = re.sub(r"(\D),(\D)", r'\1 , \2', text)
text = re.sub(r"(\d),(\D)", r'\1 , \2', text)
text = re.sub(r"(\D),(\d)", r'\1 , \2', text)

if self.language == "en":
    text = re.sub(r"([a-zA-Z])'([a-zA-Z])", r"\1 ' \2", text)
    text = re.sub(r"(\W)'([a-zA-Z])", r"\1 ' \2", text)
    text = re.sub(r"([a-zA-Z])'([a-zA-Z])", r"\1 ' \2", text)
    text = re.sub(r"([a-zA-Z])'([a-zA-Z])", r"\1 ' \2", text)
    text = re.sub(r"(\d)'(s)", r"\1 ' \2", text)
    text = re.sub(r" '\s+s ", r" 's ", text)
    text = re.sub(r" '\s+s ", r" 's ", text)

```

```

else:
    text = re.sub(r"'", r" ' ")
    text = re.sub(r"' '", r"''", text)
words = text.split()
text = ''
for i, word in enumerate(words):
    m = re.match("^(\\S+)\\.$", word)
    if m:
        pre = m.group(1)
        if ((re.search("\\.", pre) and re.search("[a-zA-Z]", pre)) or \
            (pre in self.nonbreaking_prefixes[self.language]) or \
            ((i < len(words)-1) and re.match("^\\d+", words[i+1]))):
            pass # do nothing
        elif ((pre in self.nonbreaking_prefixes_numeric[self.language] ) and \
            (i < len(words)-1) and re.match("^\\d+", words[i+1])):
            pass # do nothing
        else:
            word = pre + " ."

    text += word + " "
text = re.sub(r"'\\s+', r"''", text)

while re.search("DOTDOTMULTI", text):
    text = re.sub(r"DOTDOTMULTI", r"DOTMULTI.", text)
text = re.sub(r"DOTMULTI", r".", text)

while re.search("DASHDASHMULTI", text):
    text = re.sub(r"DASHDASHMULTI", r"DASHMULTI-", text)
text = re.sub(r"DASHMULTI", r"-", text)
text = re.sub(r"-CONTRACT-", r"''", text)

return text.split()

def tokenize_all(self, sentences, ptb=False):
    return [self.tokenize(t, ptb) for t in sentences]

if __name__ == "__main__":
    tokenizer = PTBTokenizer()
    for line in sys.stdin:
        line = line.decode("utf8")
        tokens = tokenizer.tokenize(line.strip())
        out = ' '.join(tokens)
        print(out.encode("utf8"))

```


levenshtein 矩阵图算法代码

```
def levenshtein_distance(first, second):
    lmatrix, backpointers = levenshtein_matrix(first, second)
    return lmatrix[-1][-1]

# levenshtein matrix
def levenshtein_matrix(first, second, cost_ins=1, cost_del=1, cost_sub=2):
    #if len(second) == 0 or len(second) == 0:
    #    return len(first) + len(second)
    first_length = len(first) + 1
    second_length = len(second) + 1

    distance_matrix = [[None] * second_length for x in range(first_length)]
    backpointers = {}
    distance_matrix[0][0] = 0
    for i in range(1, first_length):
        distance_matrix[i][0] = i
        edit = ("del", i-1, i, first[i-1], '', 0)
        backpointers[(i, 0)] = [(i-1,0), edit]
    for j in range(1, second_length):
        distance_matrix[0][j]=j
        edit = ("ins", 0, 0, '', second[j-1], 0) # always insert from the beginning
        #edit = ("ins", j-1, j-1, '', second[j-1], 0)
        backpointers[(0, j)] = [(0,j-1), edit]

    for i in range(1, first_length):
        for j in range(1, second_length):
            deletion = distance_matrix[i-1][j] + cost_del
            insertion = distance_matrix[i][j-1] + cost_ins
            if first[i-1] == second[j-1]:
                substitution = distance_matrix[i-1][j-1]
            else:
                substitution = distance_matrix[i-1][j-1] + cost_sub
            if substitution == min(substitution, deletion, insertion):
                distance_matrix[i][j] = substitution
                if first[i-1] != second[j-1]:
                    edit = ("sub", i-1, i, first[i-1], second[j-1], 0)
                else:
                    edit = ("noop", i-1, i, first[i-1], second[j-1], 1)
            try:
                backpointers[(i, j)].append(((i-1,j-1), edit))
            except KeyError:
                backpointers[(i, j)] = [(i-1,j-1), edit]
```

```

    if deletion == min(substitution, deletion, insertion):
        distance_matrix[i][j] = deletion
        edit = ("del", i-1, i, first[i-1], '', 0)
        try:
            backpointers[(i, j)].append(((i-1,j), edit))
        except KeyError:
            backpointers[(i, j)] = [((i-1,j), edit)]
    if insertion == min(substitution, deletion, insertion):
        distance_matrix[i][j] = insertion
        edit = ("ins", i, i, '', second[j-1], 0)
        try:
            backpointers[(i, j)].append(((i,j-1), edit))
        except KeyError:
            backpointers[(i, j)] = [((i,j-1), edit)]
    return (distance_matrix, backpointers)

```

edit_creator 编辑序列创建算法

```

for candidate, source in zip(system_read, source_read):
    if not count % 1000:
        print >> sys.stderr, count,
        count += 1
        candidate = candidate.strip()
        source = source.strip()

        candidate_tok = candidate.split()
        source_tok = source.split()
        #lmatrix, backpointers = levenshtein_matrix(source_tok, candidate_tok)
        lmatrix1, backpointers1 = levenshtein_matrix(source_tok, candidate_tok, 1, 1, 1)
        lmatrix2, backpointers2 = levenshtein_matrix(source_tok, candidate_tok, 1, 1, 2)

        #V, E, dist, edits = edit_graph(lmatrix, backpointers)
        V1, E1, dist1, edits1 = edit_graph(lmatrix1, backpointers1)
        V2, E2, dist2, edits2 = edit_graph(lmatrix2, backpointers2)

        V, E, dist, edits = merge_graph(V1, V2, E1, E2, dist1, dist2, edits1, edits2)
        V, E, dist, edits = transitive_arcs(V, E, dist, edits, max_unchanged_words,
very_verbose)

    # S = source, T = target
    print >> write_output, "S {0}".format(source)
    if verbose:
        print >> write_output, "T {0}".format(candidate)

# 最短路径搜索

```

```
gold = []
localdist = levenshtein.set_weights(E, dist, edits, gold, verbose, very_verbose)
editSeq = levenshtein.best_edit_seq_bf(V, E, localdist, edits, very_verbose)
if ignore_whitespace_casing:
    editSeq = filter(lambda x : not equals_ignore_whitespace_casing(x[2], x[3]),
editSeq)

for ed in list(reversed(editSeq)):
    if ed[2] != ed[3]:
        print >> write_output, "A {0}
{1}|||{2}|||{3}|||{4}|||{5}|||{6}".format(ed[0], ed[1], "UNK", ed[3], 'REQUIRED',
'-NONE-', 0)
        print >> write_output, ""
system_read.close()
source_read.close()
print >> sys.stderr, "Done!"
```